



- 산업 현장의 질문을 네 유형으로 나누고, RAG 시스템을 L0 L4 다섯 단계로 세우다
- 지식 원자화 · 지식 인지 과제 분해 · 훈련 가능한 분해기 — 원문 5개 질 정독
- HotpotQA·2WikiMultiHopQA·MuSiQue·LawBench 실측 해설 + 원문 그림 18종 한글 도식

질문 네 유형·시스템 다섯 단계 — PIKE-RAG(arXiv 2501.11551) 정독

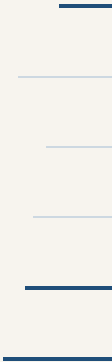
검색을 넘어 지식으로

차례

들어가며	3
01 검색만으로는 부족했다	5
02 네 가지 질문과 다섯 단계	8
03 프레임워크의 해부	12
04 L0 — 지식베이스 만들기	15
05 L1 — 사실을 가려내는 시스템	18
06 L2 — 지식을 원자로 쪼개다	23
07 L2 — 아는 만큼 나눈다	27
08 L2 — 분해기를 가르치다	29
09 L3 — 미래를 묻는 질문	32
10 L4 — 관점을 늘리는 에이전트들	35
11 벤치마크로 검증하다 — 오픈도메인	38
12 법률 벤치마크와 분해기 파인튜닝	41
13 세 가지 실패 사례와 해법	44
14 한국 IT 현장에서 읽는 PIKE-RAG	48
15 맺음말	51

DAY

들어가며



"원문: PIKE-RAG: sPeclalized KnowledgE and Rationale Augmented Generation — arXiv 2501.11551v4(2025) · Jinyu Wang·Jingjing Fu·Rui Wang·Lei Song·Jiang Bian(Microsoft Research Asia)"

"링크: <https://arxiv.org/html/2501.11551v4>"

"코드: <https://github.com/microsoft/PIKE-RAG>"

검색 증강 생성(RAG)은 대형 언어 모델이 외부 자료를 찾아 답을 만들게 하는 기법으로 널리 쓰인다. 그러나 마이크로소프트 리서치 아시아의 이 논문은 그 익숙한 그림에 정면으로 이의를 제기한다. 산업 현장의 질문은 검색만으로 풀리지 않는다는 것이다. 원천 자료는 스캔 문서, 복잡한 표, 차트, 그림까지 뒤섞여 있고, 전문 용어와 그 분야 고유의 추론 구조는 일반적인 언어 모델이 다루기 어렵다. 우편 규칙을 묻는 질문과 여러 제품을 비교하는 질문이 같은 기술로 답될 리 없는데, 기존 방법은 둘 다 한 틀에 넣고 있다. 논문은 이 격차를 지식 원천의 다양성, 도메인 특화의 결핍, 획일적 접근이라는 세 난제로 정리하고, 검색 증강을 지식과 근거의 증강으로 바꾸어야 한다고 주장한다.

제안은 두 축으로 짜였다. 하나는 질문을 난이도에 따라 네 유형으로 나누는 일이다. 사실 질문, 연결 추론 질문, 예측 질문, 창의 질문이 그것이다. 다른 하나는 시스템 능력을 다섯 단계로 세우는 일이다. 지식베이스를 구축하는 L0에서 출발해 사실 질문에 답하는 L1, 여러 근거를 잇고 추론하는 L2, 데이터를 구조화해 내다보는 L3, 창의적 해결안을 제안하는 L4로 오르는 계단이다. 각 단계마다 어떤 모듈을 더하고 어떤 난제를 푸는지가 한눈에 드러나므로, 시스템을 단계적으로 발전시킬 수 있다.

기술의 핵심은 지식 원자화와 지식 인지형 과제 분해다. 문서 청크 하나에는 여러 지식이 뒤섞여 있으니, 청크가 답할 수 있는 질문들을 뽑아내 색인으로 삼는다. 어려운 질문은 지식베이스에 무엇이 있는지 살피면서 원자 질문 여러 개로 쪼갬 다음, 답을 하나씩 모아 근거를 세워 나간다. 논문은 분해기 자체를 학습시켜 추론 비용을 줄이는 방법까지 제안한다. 이 설계는 HotpotQA, 2WikiMultiHopQA, MuSiQue 같은 멀티홉 벤치마크와 중국 법률 벤치마크 LawBench에서 일관된 성능 향상으로 이어졌다.

이 책의 구성은 세 묶음이다. 1장부터 10장까지는 논문 본문을 순서대로 정독한다. 세 난제와 질문 유형에서 출발해 프레임워크의 일곱 모듈을 해부하고, L0 지식베이스 구축부터 L1 청킹과 태깅, L2 지식 원자화와 과제 분해, 분해기 학습, L3 예측, L4 창의까지 계단 하나하나를 밟는다. 11장부터 13장까지는 평가와 사례를 읽는다. 오픈 도메인 벤치마크 세 종과 법률 벤치마크의 수치를 짚고, 단일 경로 방식이 실패하는 세 사례를 대조한다. 원문의 표는 열이 많아 지면에 그대로 옮기기 어려우므로, 이 책은 논의에 필요한 핵심 열만 남겨 한국어 표로 다시 만들었고 본문에서 나머지 수치를 서사로 풀었다. 14장은 한국 IT 현장에서 읽는 각색이고 15장이 맺음말이다.

그림은 원문의 Figure 1부터 18까지를 한글로 재구성한 도식 열여덟 장을 쓴다. 캡션 번호를 원문 그림 번호와 같게 두었으므로, 원문을 펴고 그대로 대조하며 읽을 수 있다.

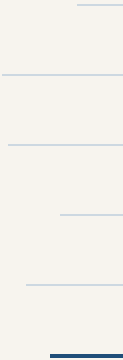
각색에 대하여 밝혀둔다. 14장은 필자가 원문 개념을 한국의 개발 현장 언어로 옮긴 해설이며 원문 저자의 주장이 아니다. 그 밖의 장은 모두 논문이 말하는 바를 충실히 다루고, 논문에서 확인한 수치만 인용했다.

시간이 모자란 독자는 2장에서 질문 유형과 다섯 단계의 전체 지도를, 6장과 7장에서 지식 원자화와 과제 분해라는 기술 핵심을, 11장에서 벤치마크 결과를 읽고 나머지를 채우면 된다. 짧은 경로에서는 그림 1·10·11이 핵심 이정표가 된다.

DAY

01

검색만으로는 부족했다



첫봇에게 어제 뉴스 요약 부탁하면 그럴듯한 답이 돌아온다. 그러나 반도체 데이터시트에서 물성치를 확인하거나, 두 약품의 심사 절차를 항목별로 비교하라고 하면 이야기가 달라진다. 논문은 바로 이 경계선에서 출발한다. 외부 자료를 검색해 붙이는 기존 검색 증강 생성(RAG) 방식이 산업 현장의 전문 질의 앞에서 왜 무너지는지를 해부하고, 무엇을 보강해야 하는지를 차근차근 정리한다.

I 대형 언어 모델의 네 가지 균열

대형 언어 모델(LLM)은 방대한 말뭉치를 학습해 문장 완성부터 번역과 요약까지 폭넓은 언어 과제를 해냈다. 그러나 저자들은 전문 분야의 질의 앞에서 뚜렷이 드러나는 약점을 네 가지로 짚는다. 첫째는 도메인 특화 질의 취약이다. 전문 분야의 학습 자료가 희소하고 해당 분야의 지식과 근거 구조에 대한 이해가 얕아, 모델은 틀린 답은 물론 전문가 수준의 정밀함을 갖추지 못한 답을 내놓는다. 둘째는 환각(hallucination)이다. 모델이 사실에 근거하지 않은 정보를 지어내는 이 현상은 신뢰가 생명인 산업 응용에서 치명적인 결함으로 작동한다. 셋째는 지식 시점의 고정이다. 학습이 마지막으로 이루어진 시점에서 지식이 굳어 버리는 시간적 정체(temporal stasis)를 모델 스스로 벗어나지 못한다. 넷째는 장문 이해의 약화다. 긴 맥락 속에서 과제의 정의를 끝까지 붙잡지 못하고, 과제가 복잡하고 까다로워질수록 성능이 크게 떨어진다.

RAG는 이 네 균열을 메우기 위해 제안된 조합이다. 생성 능력은 모델에 두고 사실성은 검색으로 담보한다는 발상이다. 생성된 내용을 사실 데이터에 고정시켜 정확도와 신뢰도를 함께 끌어올려 LLM의 산업 현장 실용화에 길을 열었다. 그러나 기존 RAG는 텍스트 검색과 모델의 이해력에 크게 기울어 있고, 다양한 원천 자료에서 지식을 뽑아 이해하고 활용하는 일에는 관심이 얕았다.

I 산업용 RAG를 가로막는 세 가지 난제

논문은 RAG가 산업 현장에서 마주하는 어려움을 세 가지로 압축한다.

첫째는 지식 원천의 다양성(knowledge source diversity)이다. 산업용 RAG 시스템은 여러 해에 걸쳐 여러 분야에서 모은 방대한 원천 문서 위에 세워진다. 스캔 이미지, 디지털 텍스트 파일, 웹 데이터는 물론 전문 데이터베이스까지 뒤섞인다. 널리 쓰이는 벤치마크 데이터셋이 미리 잘라 둔 단순한 말뭉치를 내놓는 것과 대조적이다. 저자들은 LED 제품 데이터시트를 예다. 성능 특성은 복잡한 표로, 전기적 특성은 차트로, 설치 방법은 그림으로 담겨 있다. 텍스트 바깥의 지식을 묻는 질의 앞에서 기존 RAG가 크게 당황하는 지점이다.

둘째는 도메인 특화의 결핍(domain specialization deficit)이다. 산업 응용은 전문 분야 고유의 용어와 전문성, 그리고 그 분야만의 논리 틀을 다뤄야 한다. 범용 지식 중심 데이터셋으로 만든 RAG는 이런 필드에서 기대에 미치지 못한다. 반도체 설계를 보라. 연구는 물리적 물성에 대한 깊은 이해에 기대는데, 모델이 연구 문서에서 전문 지식과 근거를 뽑아 조직하도록 놓으면 필수적인 물리 원리를 제대로 붙잡지 못한다. 그 결과 결정적 문제 요소에 대한 해석이 불완전해지고, 물리 원리에 뿌리내린 근거가 빠진 답이 나온다. 전문 콘텐츠 생성의 품질을 평가하는 일 자체도 어려워, 알고리즘 개발과 실무 배치 양쪽이 발목을 잡힌다.

셋째는 획일적 접근(one-size-fits-all)이다. RAG 응용 시나리오는 비슷한 틀을 공유하면서도 요구 능력이 제각각이다. 우편물 발송 요건처럼 규칙을 묻는 시나리오에서는 질의와 규칙 사이의 의미 간극을 메워 관련 사실 규칙을 찾아내는 능력이 관건이다. 여러 제품을 여러 항목에서 비교하는 멀티홉 시나리오에서는 여러 원천에서 정보를 뽑아 다단계 추론을 수행하는 능력이 필요하다. 한 시나리오 안에서도 난이도가 갈린다. 그러나 기존 접근은 시나리오 안팎의 복잡도와 특수 요구를 가리지 않고 한 가지 전략을 밀어 붙였고, 실무가 요구하는 포괄적 정확도에 미치지 못했다.

I 관련 연구의 지형

RAG 연구는 크게 세 갈래로 정리된다. 나이브(naive) RAG는 원시 데이터를 균일한 평문으로 바꾸어 작은 청크로 자르고, 벡터 공간에 박아 질의 기반 검색으로 상위 관련 청크를 프롬프트에 붙인다. 어드밴스드(advanced) RAG는 검색 전, 검색, 검색 후 과정에 질의 최적화, 다중 입자도 청킹(multi-granularity chunking), 혼합 검색, 청크 재순위화 같은 보강을

없어 검색 품질을 끌어올린다. 모듈러(modular) RAG는 파이프라인과 시스템 모듈에 정교한 개선을 더해 더 다양한 원천 데이터를 다루고, 원시 정보를 구조화된 데이터를 거쳐 지식으로 바꾼다.

복잡한 과제를 겨냥한 조율 방식도 잇달았다. ITERRETGEN과 DSP는 검색과 읽기를 되풀이하면서 이번 턴의 생성 결과를 다음 턴 검색의 맥락으로 쓴다. FLARE는 신뢰도 기반 능동 검색으로, 다시 만든 문장의 낮은 신뢰도 토큰을 보며 질의를 동적으로 조정한다. 이런 루프형 파이프라인은 정답을 향해 점차 수렴하며 다양한 요구에 유연하게 대응한다.

지식베이스를 풍부하게 하려는 흐름도 있다. 평문 검색은 정보가 잡음으로 덮여 검색 품질이 떨어지고, 복잡한 문제가 요구하는 객체 간 관계를 담아내지 못한다. GraphRAG는 지식그래프 생성과 질의 중심 요약물 RAG와 결합해 국소 질문과 전역 질문을 함께 다룬다. HOLMES는 초관계 지식그래프를 만들고 가지치기해 정제 그래프로 만든 뒤 멀티홉 질의응답의 입력으로 쓴다. 다만 저자들은 한 가지를 못박는다. 지식그래프 구축은 자원이 극도로 많이 들고, 비용이 말뚝치 크기에 비례해 불어난다.

멀티홉 질의응답(Multi-hop Question Answering, MHQA) 계보도 이 논문의 배경이다. 전통적인 그래프 기반 방법은 그래프를 만들고 그래프 신경망으로 추론했으나, LLM 시대에는 지식그래프를 만들어 검색하고 LLM으로 답을 생성하는 쪽으로 진화했다. 다른 갈래는 이전 답에 기대어 다음 질문을 만들어 멀티홉 질문을 하위 질의 열로 바꾼다. Self-RAG는 검색과 생성, 비평을 배운 모델로 문제를 푸는 지도 학습 접근이고, beam-retrieval은 모든 홉에서 인코더와 분류 헤드를 함께 최적화해 멀티홉 검색을 종단간으로 모델링한다. Self-Ask는 본 답을 내기 전에 후속 질문을 스스로 던지게 해 질문 자동 분해를 검색과 매끄럽게 엮는다.

■ 논문이 내놓은 네 가지 기여

첫째, 저자는 전문 지식과 근거가 증강의 핵심 토대가 되어야 한다고 제안한다. 지식 추출과 이해, 활용의 난이도에 따라 과제를 서로 다른 유형으로 분류하는 새 패러다임이 그 산물이며, 이 분류는 시스템 능력을 층위로 나누어 단계적 개발을 떠받친다. 둘째, 전문 지식 추출과 근거 구성에 초점을 맞춘 PIKE-RAG 프레임워크를 소개한다. 능력 요구 수준에 맞춰 시스템 틀을 맞춤 제작할 수 있어 다용성이 뛰어나다. 셋째, 지식 원자화(knowledge atomizing)와 지식 인지형 과제 분해(knowledge-aware task decomposition)로 멀티홉 벤치마크, 특히 두 홉이 넘는 시나리오에서 뚜렷한 향상을 냈다. 넷째, 근거 중심 데이터를 수집해 분해기를 학습시키는 전략으로, 도메인 특유의 근거를 과제 분해와 답 탐색 경로 양쪽에 심었다.

여기까지가 문제의 지도다. 그러나 난제를 정리하는 것과 푸는 것은 별개의 일이다. 다음 장에서 저자가 이 문제들을 어떤 언어로 다시 세웠는지, 질문의 유형과 시스템의 단계라는 두 축으로 풀어 본다.

DAY

02

네 가지 질문과 다섯 단계



앞 장에서 본 세 난제를 정면으로 받아들이려면 문제를 다시 세우는 언어가 필요하다. 저자는 RAG 프레임워크를 지식베이스, 과제 분류, 시스템 개발의 세 관점에서 개념화한다. 지식베이스가 검색과 생성의 초석이고, 과제와 시스템은 능력 요구에 따라 복잡도와 단계가 갈린다. 이 장은 과제 분류와 시스템 단계를 읽는다.

지식베이스라는 이름의 그래프

산업 현장의 전문 지식은 제조, 에너지, 물류 같은 분야에서 수년간 쌓인 데이터에서 나온다. 제약 산업의 원천은 연구개발 문서와 수년치 의약품 허가 파일인데, 표와 차트, 그림 같은 다중 양상 콘텐츠가 문제 해결의 열쇠로 박혀 있고, 하이퍼링크와 참조 같은 기능적 연결이 지식의 논리적 조직을 드러낸다. 저자는 이 지식베이스를 다층 이종 그래프(multi-layer heterogeneous graph) G 로 구조화할 것을 제안한다. 노드 집합 V 와 에지 집합 E 로 이루어진 그래프의 내부 구조는 뒤에서 다시 해부하고, 여기서는 설계가 검색 방법과 성능을 직접 좌우한다는 사실만 붙잡아 둔다.

과제 난이도를 가르는 네 요인

과제의 난이도는 몇 가지 결정적 요인과 맞닿아 있다.

- 지식의 관련성과 완전성(relevance and completeness of knowledge): 필요한 정보가 지식베이스 안에 얼마나 존재하고, 주제를 얼마나 고루 다루는가.
- 지식 추출의 복잡도(complexity of knowledge extraction): 흩어진 원천에 있거나 텍스트에 암묵적으로 박힌 관련 지식 조각들을 모두 정확히 꺼내는 일이 얼마나 어려운가.
- 이해와 추론의 깊이(depth of understanding and reasoning): 검색해 낸 정보를 이해하고 연결하며 여러 단계 추론을 수행하는 데 얼마나 많은 인지 처리가 필요한가.
- 지식 활용의 효과(effectiveness of knowledge utilization): 추출한 지식을 종합하고 조직해 통찰이나 예측을 만드는 적용 단계가 얼마나 정교해야 하는가.

질문의 네 유형

이 네 요인을 저울에 올려, 저자는 산업 현장의 질문을 난이도에 따라 네 유형으로 나눈다.

사실 질문(factual questions)은 말뭉치에 명시적으로 놓인 구체적 정보를 직접 꺼내는 일이다. 참조 텍스트를 대화 맥락에서 처리할 수 있고, 관련 사실을 검색으로 찾아내면 답이 완성된다. 연결 추론 질문(linkable-reasoning questions)은 한 단계 깊다. 여러 원천에서 관련 정보를 모으거나 여러 단계 추론을 수행해야 하고, 답이 여러 텍스트에 암묵적으로 흩어져 있다. 연결과 추론 과정의 차이에 따라 넷으로 다시 갈린다. 여러 개체를 순서대로 잇는 연결(bridging) 질문, 두 개체의 지정 속성을 나란히 놓는 비교(comparative) 질문, 검색 데이터 위에 통계 분석을 얹는 정량(quantitative) 질문, 여러 원천이나 대량 텍스트를 압축해 하나의 요약으로 엮는 요약(summarizing) 질문이 그것이다. 요약 질문은 다른 유형의 요소를 결합하는 경우가 잦아, 합리적인 답 탐색 경로를 세우는 일이 관건이다. 예측 질문(predictive questions)은 답이 원문에 없다. 기존 사실 위에 귀납적 추론을 얹어 미래를 내다보는데, 시계열처럼 분석 가능한 형태로 지식을 조직해야 한다. 창의 질문(creative questions)은 전문 분야 고유의 논리와 창의적 문제 해결을 동원한다. 패턴과 영향 요인을 찾아 새로운 해법의 실마리를 제공하는 것이 목표이지, 바로 실행 가능한 해결안을 내놓는 것이 아니다.

저자가 FDA가 공개한 데이터로 제약 응용 전문가들과 함께 만든 세 질문이 이 분류의 힘을 보여 준다. 생물유사성의약품(biosimilar)과 그 승인일을 묻는 Q1은 모든 제품과 승인일을 표 한 장에 정리해 둔 덕에 명시적 정보만으로 끝나는 사실 질문이다. 상호교환 가능한 생물유사성의약품의 총수를 묻는 Q2는 단일 출처를 참조하는 것으로 끝나지 않는다. 조건에 맞는 제품을 모두 찾은 뒤 합산하는 통계 집계 단계가 더 필요해, 중간 처리를 요하는 연결 추론 질문이다. 앞으로의 상황을 묻는 Q3는 답이 지식베이스에 아예 존재하지 않는다. 관련 데이터를 모아 숨은 패턴을 추론하고 그 규칙 위에서 예측해야 하므로 예측 질문이다. 겉보기에 비슷한 세 질문이 서로 다른 유형으로 갈라지는 이유는 하나다. 질문의 유형은 질문의 문구가 아니라 지식베이스의 상태가 정한다.



그림 1 질문 유형에 따라 달라지는 해결 방법

그림 1은 네 유형의 질문과 각각의 예를 나란히 놓는다. 화면 왼쪽의 사실 질문이 표에서 바로 답을 고르는 데 견주어, 연결 추론 질문은 연결·비교·정량·요약 네 갈래로 갈라지고 예측 질문은 과거 근거를 구조화한 뒤 앞을 내다본다. 유형마다 지원 데이터와 기대되는 추론 과정이 다르다는 점이 그림의 요지다.

시스템 단계 L1부터 L4까지

질문 유형이 정해졌으니 시스템 능력도 그 자에 맞춰 세울 수 있다. 저자는 네 유형의 문제 해결 능력에 따라 RAG 시스템을 네 단계로 분류한다. 원문 표 1을 한국어로 옮기면 다음과 같다.

레벨	능력 설명
L1	사실 질문에 정확하고 믿을 만한 답을 제공해 기본 정보 검색의 튼튼한 토대를 만든다.
L2	사실 질문과 연결 추론 질문을 모두 정확하고 믿을 만하게 다루어, 더 복잡한 다단계 검색과 추론 과제를 수행한다.
L3	사실 질문과 연결 추론 질문에서 정확성과 신뢰성을 유지하면서, 예측 질문에 그럴듯한 예측을 내놓는 능력을 더한다.
L4	창의 질문에 근거가 탄탄한 계획이나 해결안을 제안하면서, 예측 질문에 대한 그럴듯한 예측 능력과 사실 질문, 연결 추론 질문에 대한 정확하고 믿을 만한 답변 능력도 유지한다.

표 밖에서 한 가지를 덧붙인다. 논문은 지식베이스 구축 단계를 시스템 개발의 L0 단계로 지정한다. 즉 사다리의 바닥은 구축이며, 올라갈수록 이해와 추론, 혁신의 수위가 높아진다.

레벨별로 다른 난제

단계마다 시스템이 부딪히는 벽도 달라진다. 원문 표 2가 정리한 단계별 난제를 풀어 쓰면 다음과 같다.

- L0에서는 원천 문서 형식이 제각각이라 지식 추출이 어렵고, 정교한 파일 파싱 기술이 요구된다. 이질 데이터에서 고품질 지식베이스를 만드는 일에 지식 조직과 통합의 복잡도가 몰린다.

- L1에서는 잘못된 청킹이 의미 연속성을 깨뜨려 지식 이해와 추출, 정확한 검색을 가로막는다. 임베딩 모델이 전문 용어와 별칭을 제대로 정렬하지 못해 검색 정밀도가 떨어진다.
- L2에서는 청크에 관련 정보와 무관한 정보가 뒤섞여 고품질 데이터 검색이 정확한 생성의 전제가 된다. 과제와 그 이면의 근거를 이해하고 분해하는 일이 지원 데이터의 가용성을 외면한 채 모델 능력에만 기댄다.
- L3에서는 예측 추론을 떠받칠 지식 수집과 조직이 화두다. 모델이 전문 추론 논리를 적용하는 데 한계가 있어 예측 효과가 제한된다.
- L4에서는 요인 간 의존이 얽혀 해가 유일하지 않은 지식베이스에서 일관된 논리 근거를 뽑아내기가 어렵다. 창의 질문의 개방성 때문에 추론과 지식 종합을 정량 평가하기도 어렵다.

질문 유형과 시스템 단계라는 지도가 마련되었다. 다음 장은 이 지도 위에서 실제로 움직이는 기계를 연다. 일곱 개의 모듈과 그 모듈들을 잇는 순환이 PIKE-RAG 프레임워크를 어떻게 이루는지 해부한다.

DAY

03

프레임워크의 해부



지도가 있으면 이제 기계를 볼 차례다. 저자는 지식베이스, 과제 분류, 시스템 단계라는 정식화 위에 범용적이고 확장 가능한 RAG 프레임워크를 올린다. 이 프레임워크의 매력은 시스템 단계가 오를 때 주요 모듈 안의 서브모듈을 조정하는 것만으로 발전을 이룰 수 있다는 점이다. 거대한 재설계가 아니라 부품 교환으로 계단을 오르는 설계다.

일곱 개의 모듈

프레임워크의 몸체는 일곱 개의 기본 모듈로 이루어진다. 다양한 형식의 도메인 문서를 기계가 읽을 수 있는 형식으로 바꾸는 파일 파싱(file parsing), 청크를 만들고 지식 단위를 뽑아내는 지식 추출(knowledge extraction), 추출한 지식을 여러 구조화된 형식으로 보관하는 지식 저장(knowledge storage), 하이브리드 검색 전략으로 관련 정보에 접근하는 지식 검색(knowledge retrieval), 검색해 온 정보를 가다듬어 조직하는 지식 조직(knowledge organization), 조직된 지식 위에서 추론을 수행하는 지식 중심 추론(knowledge-centric reasoning), 그리고 검색과 추론의 경로를 동적으로 관리하는 과제 분해와 조율(task decomposition and coordination)이 그 목록이다. 파일 파싱 모듈이 문서를 파일 단위로 쪼개 정보 자원 계층의 그래프를 세우면, 지식 추출 모듈이 텍스트를 청킹해 코퍼스 계층과 정제 지식 계층의 그래프를 만든다. 세워진 이중 그래프가 지식베이스가 되어 검색의 대상이 된다.



그림 2 PIKE-RAG 전체 프레임워크

그림 2는 이 일곱 모듈이 흐름 위에 어떻게 배열되는지 보여 준다. 왼쪽에서 원시 문서가 들어와 파싱과 추출을 거쳐 지식베이스에 쌓이고, 검색과 조직을 지나 추론으로 이어진다. 한 가지 눈여겨 볼 점은 되먹임 고리다. 조직되고 검증된 지식이 다시 지식베이스를 정제하고 보강하는 데 쓰인다. 지식베이스는 모으기만 하는 창고가 아니라 쓰일수록 좋아지는 살아 있는 구조물이라는 뜻이다.

되풀이되는 검색과 생성

질문 유형마다 답 탐색의 근거 경로가 다르다는 것은 앞에서 확인했다. 관련 정보의 가용 여부, 지식 추출의 복잡도, 추론의 정교함이 경로를 바꾼다. 그렇다면 한 번의 검색과 한 번의 생성으로 끝나는 단일 패스로는 이 질문들을 감당하기 어렵다. 저자의 대답은 과제 분해와 조율의 감독 아래 도는 반복 검색-생성(iterative retrieval-generation) 메커니즘이다. 산업 응용의 질문이 과제 분해 모듈에 들어가면 예비 분해안이 나온다. 이 안에는 검색 단계와 추론 단계, 그 밖에 필요한 작업들

이 적혀 있다. 지침을 따라 지식 검색 모듈이 관련 정보를 모으면 지식 조직 모듈이 이를 처리하고 조직하고, 조직된 지식으로 지식 중심 추론을 수행해 중간 답을 얻는다. 그러면 새로 모은 정보와 중간 답을 반영해 과제 분해 모듈이 다음 반복을 위한 갱신된 분해안을 다시 만든다. 이 고리는 관련 정보를 조금씩 모으고 점층적으로 늘어나는 맥락 위에서 추론을 이어가게 하여, 더 정확하고 포괄적인 답으로 향한다.

단계별로 자라나는 시스템

프레임워크가 확장 가능하다는 말은 단계 사이에 상속 관계가 있다는 뜻이다. L0에서 L4로 오르는 동안 상위 단계 시스템은 하위 단계의 모듈을 물려받고 능력을 보강하는 새 모듈을 얻는다. L1에 비해 L2는 반복 검색-생성 경로를 활용하는 과제 분해와 조율 모듈을 새로 도입하는 동시에, 정제 지식 생성 같은 더 진보한 지식 추출 모듈을 갖춘다. L3는 예측 질문의 비중이 커지면서 지식 조직과 추론 양쪽에 높은 요구를 걸어, 지식 조직 모듈에 지식 구조화(knowledge structuring)와 지식 유도(knowledge induction) 서브모듈을 더하고 지식 중심 추론 모듈에 예측(forecasting) 서브모듈을 확장한다. L4는 이미 갖춘 지식베이스에서 복잡한 근거를 뽑아내는 일이 워낙 어려워, 여러 관점에서 추론을 일으키는 멀티 에이전트 플래닝(multi-agent planning) 모듈을 도입한다.



그림 3 지식베이스를 구성하는 세 개의 계층

그림 3은 프레임워크의 밑바닥을 이루는 지식베이스의 내부를 드러낸다. 정보 자원 계층, 코퍼스 계층, 정제 지식 계층의 세 계층이 서로 다른 처리 단계, 서로 다른 입자도와 추상화 수준을 담당한다. 모듈 이야기를 시작하기 전에 이 세 층이 무엇을 담는지부터 뜯어볼 필요가 있다. 다음 장은 사다리의 바닥인 L0, 즉 이 세 계층의 지식베이스를 어떻게 구축하는지를 읽는다.

DAY

04

LO — 지식베이스 만들기



아무리 똑똑한 검색 알고리즘도 담아 둘 지식이 엉망이면 빛을 내지 못한다. PIKE-RAG는 시스템 발전의 출발점을 검색 기술이 아닌 지식베이스 구축에 두고, 이 단계를 L0라 부른다. L0의 목표는 단순하다. 도메인 문서를 기계가 읽을 수 있는 형태로 바꾸고, 앞 장에서 예고한 다층 이중 그래프를 여기서 실제로 쌓는다. 이 그래프가 이후 모든 상위 단계의 추론과 검색이 서는 뼈대다.

파일 파싱

지식베이스를 만드는 첫 걸음은 여러 형식의 파일을 읽어 내는 일이다. LangChain 같은 프레임워크는 비정형 텍스트 문서를 해석하고 정보를 뽑아 내는 도구 모음을 제공한다. 스캔한 문서나 이미지는 광학 문자 인식(OCR)과 표 추출에 특화된 딥러닝 도구, 상용 클라우드 API가 구조화된 텍스트로 바뀌 준다.

그러나 도메인 문서는 보통 복잡한 표와 차트, 그림을 함께 품는다. 이를 무조건 텍스트로만 변환하면 정보가 잘리고 원래의 논리 구조가 깨진다. 그래서 L0는 두 가지를 추가한다. 첫째, 레이아웃 분석을 통해 문서의 배치 정보를 살린다. 이 정보는 이후 청킹 작업에서 청크가 온전한 단위로 잘리도록 돕는다. 둘째, 차트와 그림 같은 멀티모달 요소를 없애지 않고 보존한다. 그림과 차트는 비전언어모델(VLM)로 서술한 설명 텍스트를 붙여 검색 대상에 포함시킨다. 원문이 지닌 온전함과 풍부함을 지식베이스까지 그대로 옮기는 장치다.

세 층으로 쌓는 그래프

파싱을 마친 정보는 다층 이질적 그래프에 자리 잡는다. 이 그래프는 문서, 섹션, 청크, 그림, 표 사이의 관계를 노드와 에지로 표현하며 세 층으로 구성된다.

정보 자원층은 문서나 데이터베이스 같은 다양한 정보원을 소스 노드로 올린다. 노드 사이를 잇는 에지는 하이퍼링크, 인용, 데이터베이스 링크 같은 참조 관계를 담는다. 덕분에 여러 출처를 오가며 근거를 대조하는 추론의 토대가 마련된다.

코퍼스층은 파싱 결과를 섹션과 청크 단위로 정리하면서도 문서 원래의 계층 구조를 유지한다. 표와 그림 같은 멀티모달 내용은 LLM이 요약한 청크 노드로 통합되어 검색 대상에 끼어든다. 한 청크 단위부터 문서 전체까지 다양한 입자도로 지식을 다룰 수 있게 해 주는 층이다.

정제 지식층은 코퍼스를 한 번 더 가공해 구조화된 지식으로 응축한다. 여기서 뽑아 낸 지식은 개체와 논리 관계를 명시적으로 담고 있어, 코퍼스층만으로는 불가능한 심층 추론을 가능하게 한다.

지식 종류의 세 가지 형태

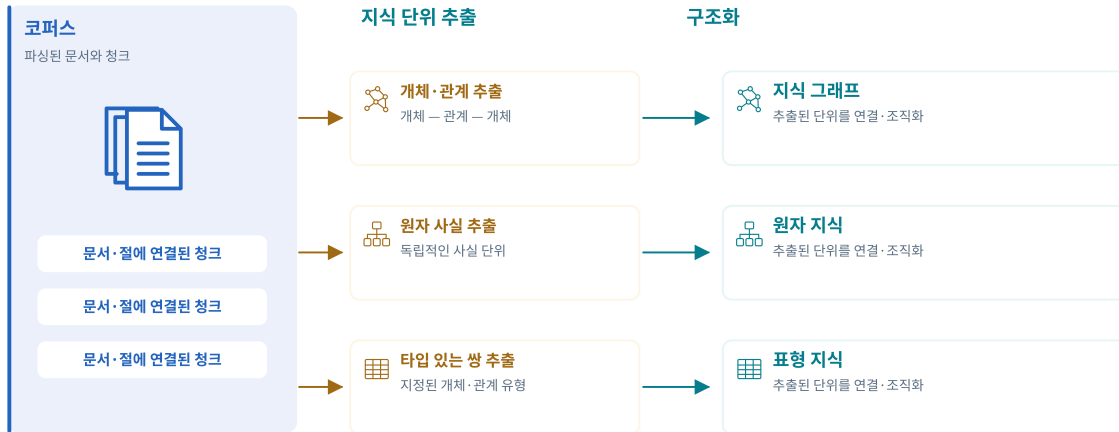
코퍼스 텍스트에서 정제 지식층을 만드는 과정을 지식 종류라 부른다. 개체명 인식(NER)과 관계 추출 기법이 이 작업을 이끈다.

PIKE-RAG / 한국어 그림 해설

04 / 18

코퍼스에서 정제 지식을 만드는 과정

문장을 그대로 쌓는 대신, 추출 목적에 맞는 지식 단위로 정리



여기서 정제는 문서의 지식을 구조화하는 과정이며, 언어모델 압축을 뜻하지 않습니다.

출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 4 / PDF 12쪽

한글 재구성 · 원문 번호 유지

그림 4 코퍼스에서 정제 지식을 만드는 과정

그림 4는 증류의 전체 흐름을 보여 준다. 코퍼스 텍스트에 맞춤 추출 패턴을 적용해 지식 단위를 뽑아 내고, 이 단위들을 정제 지식층에 구조화된 지식으로 모은다. 완성된 지식의 모습은 대표적으로 세 가지다.

첫째는 지식그래프다. LLM이 코퍼스 텍스트에서 개체와 개체 사이의 관계를 추출해 노드-에지-노드 구조의 지식 단위를 만든다. 이 단위들을 모두 합치면 하나의 그래프가 된다.

둘째는 원자 지식이다. 코퍼스 텍스트를 더 이상 쪼갤 수 없는 원자 문장들의 집합으로 분할하고, 각 문장을 지식 단위로 삼는다. 여기에 코퍼스 노드 사이의 관계를 결합해 완성한다.

셋째는 표 형태 지식이다. 지정한 유형과 관계를 지닌 개체쌍을 텍스트에서 추출해 지식 단위로 쓰고, 개체쌍을 모아 표로 조립한다. 속성을 나열해 비교해야 하는 질문에서 특히 힘을 발휘한다.

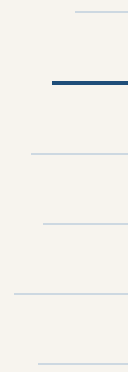
기반 위에서 시작하는 검색

지식베이스가 아무리 탄탄해도 스스로 답을 찾아 주지는 않는다. 저장된 지식을 꺼내 쓰는 검색과 생성 능력이 뒤따라야 한다. PIKE-RAG는 이 능력을 붙인 첫 단계를 L1이라 부르고, 사실 확인형 질문을 정면으로 다룬다. 다음 장에서는 L1이 청킹과 용어의 벽을 어떻게 허무는지 살펴본다.

DAY

05

L1 — 사실을 가려내는 시스템



지식베이스를 쌓았으니 이제 질문에 답할 차례다. L1 시스템이 노리는 것은 명확하다. 코퍼스 어딘가에 답이 분명히 있는 사실형 질문을 정확히 가려 내는 것이다. 그런데 여기서 두 가지 벽이 나타난다. 하나는 청킹이, 다른 하나는 용어가 막는 벽이다.

청킹이 깨는 응집성

첫 번째 난제는 청킹 자체에서 온다. 청크는 한편으로는 벡터화해 데이터베이스에 저장하는 검색의 단위고, 다른 한편으로는 지식 추출과 요약의 원천이 된다. 그래서 청킹이 어긋나면 문제가 이중으로 번진다. 벡터에 필요한 의미가 담기지 않을 뿐 아니라, 온전한 문맥이 있어야 가능한 지식 추출까지 막힌다.

법규 텍스트가 대표적인 반례다. 고정 크기 청킹은 법 조문의 문맥을 무자비하게 자르고, 조건을 누락시키기 쉽다. 어떤 규정의 적용 요건이 문장 여러 개에 걸쳐 있는데 경계에서 잘리면, 나중에 그 청크만 읽은 시스템은 조건이 없는 규정이라고 믿게 된다. 이는 이후 지식 추출의 품질과 정확도를 함께 갇아 먹는다.

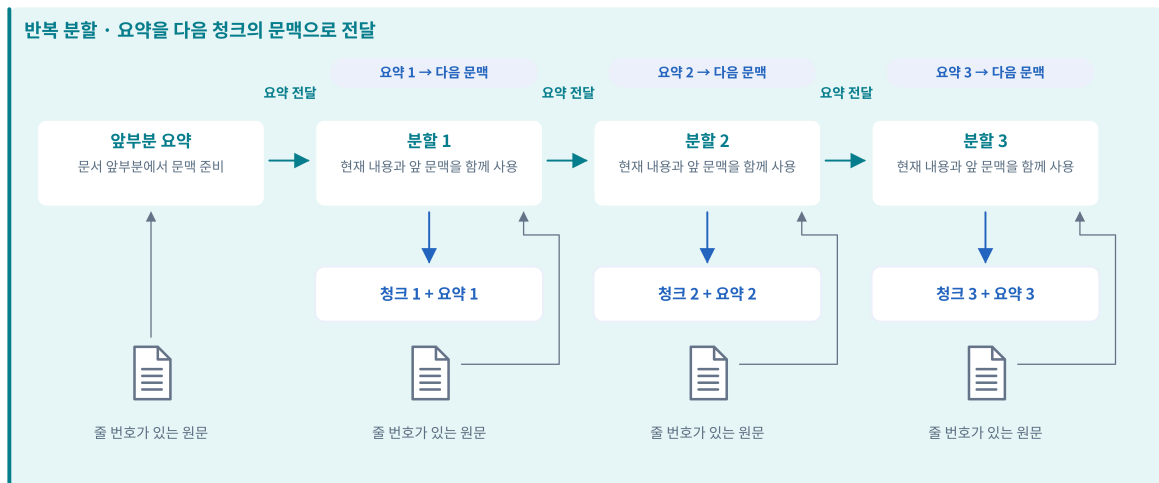
PIKE-RAG는 이를 막기 위해 반복 텍스트 분할(recurrent text splitting)을 제안한다. 원본 텍스트를 순서대로 나누되, 앞 청크에서 만든 앞 요약(forward summary)을 다음 청크의 요약 생성에 물려 주는 알고리즘이다. 각 청크는 앞 요약과 현재 청크를 함께 담은 프롬프트 템플릿으로 요약되고, 요약은 청크 끝에 함께 저장된다. 처리한 청크를 빼고 앞 요약을 갱신하는 일을 텍스트가 끝날 때까지 반복하므로, 청크마다 앞선 맥락이 끊기지 않고 이어진다. 필요하면 청크 크기를 텍스트 내용과 구조에 맞춰 동적으로 조정하기도 한다.

PIKE-RAG / 한국어 그림 해설

05 / 18

앞 문맥을 이어 가는 반복 청크 분할

앞부분의 요약을 다음 분할에 전달하여 문서의 흐름을 유지



처리한 부분을 제거하고 요약을 갱신하며 반복합니다. 청크의 크기는 내용·구조에 맞춰 조정할 수 있습니다.

출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 5 / PDF 13쪽

한글 재구성 · 원문 번호 유지

그림 5 앞 문맥을 이어 가는 반복 청크 분할

그림 5는 이 분할 과정을 도식화한다. 청크가 넘어갈 때마다 앞 요약이 다음 요약의 입력으로 흘러 들어가는 모습이 핵심이다. 경계에서 잘려도 맥락이 요약 형태로 살아 남으므로, 검색 단위와 지식 추출 원천으로서의 품질을 동시에 지킨다.

L1 프레임워크의 조립

이렇게 다듬은 부품들을 조립하면 L1의 전체 그림이 나온다.

PIKE-RAG / 한국어 그림 해설

06 / 18

L1 · 사실 확인을 위한 RAG

문서에 명시된 사실을 찾고, 검색한 근거에 기반해 답변



출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 6 / PDF 13쪽

한글 재구성 · 원문 번호 유지

그림 6 L1 · 사실 확인을 위한 RAG

그림 6은 L1 프레임워크의 전체 조립도다. L0에서 만든 지식베이스 위에 검색과 지식 정리 모듈을 얹고, 과제 분해와 조율, 지식 정리의 초기 단계 구성 요소를 더해 더 복잡한 질문을 감당한다. 여기서 강화된 청킹과 자동 태깅 하위 모듈이 지식 추출 모듈 안에 들어간다. 청킹은 앞서 본 반복 분할로 완성했고, 남은 것은 용어의 벽이다.

자동 태깅으로 용어 격차를 메우다

두 번째 난제는 질문과 코퍼스가 쓰는 언어가 다르다는 점이다. 도메인 코퍼스는 격식을 갖춘 전문적인 표현으로 쓰여 있지만, 사용자의 질문은 일상어로 풀어 말한다. 의료 질의응답(medQA) 과제를 보면 뚜렷하다. 질문은 증상을 대화체로 서술하는 반면, 코퍼스 속 의학 지식은 전문 용어로 적혀 있다. 임베딩 모델만으로는 이 도메인 격차를 좁히기 어렵고, 청크 검색 정확도가 떨어진다.

자동 태깅(auto-tagging) 모듈이 이 간극을 메운다. 절차는 이렇다. 먼저 LLM으로 코퍼스 청크의 핵심 요인을 찾아 일반화한 범주 이름, 곧 태그 클래스를 정의한다. 태그 클래스를 바탕으로 의미 태그 추출 프롬프트를 만들고, 코퍼스 전체에서 태그를 뽑아 코퍼스 태그 집합을 구성한다. 실제 QA 표본까지 손에 있다면 한 단계가 더해진다. 질문과 그에 대응하는 정답 청크 각각에서 태그를 추출한 뒤, 두 태그 집합을 LLM으로 연결해 태그 쌍 집합을 만든다.

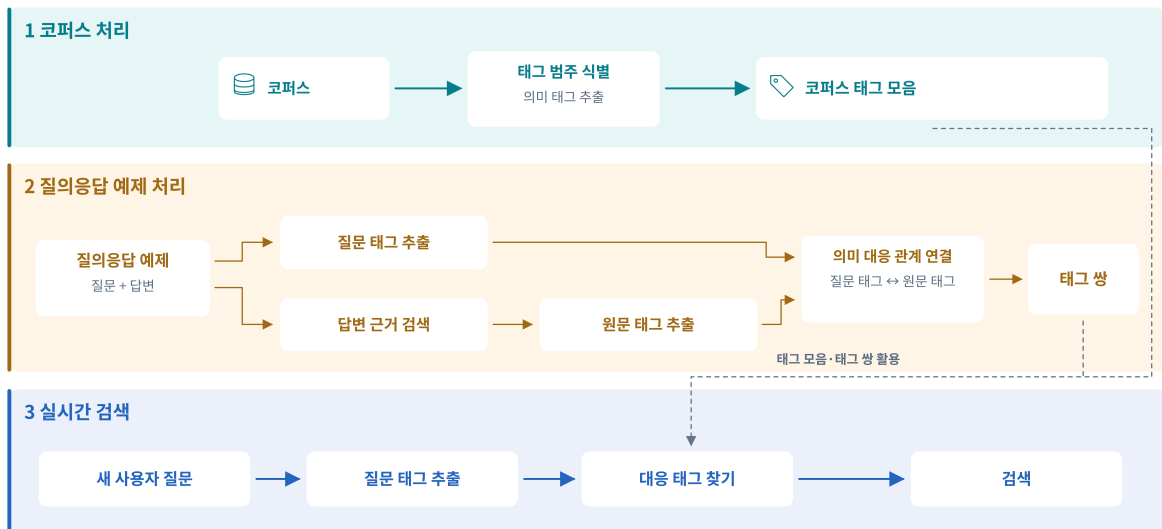
검색 시점에는 질문에서 태그를 뽑아 태그 집합과 태그 쌍 집합을 통해 코퍼스 도메인의 태그로 사상한다. 이 사상 결과는 질문 재작성이나 키워드 검색에 쓰인다. 환자가 “숨이 가빠요”라고 말한 것을 코퍼스의 호흡곤란이라는 용어로 이어 주는 통역 역할이며, 검색의 재현율과 정밀도를 함께 끌어 올린다.

PIKE-RAG / 한국어 그림 해설

07 / 18

자동 태깅으로 질문과 전문 용어 연결

문서의 전문적 표현과 사용자의 질문 표현을 의미 태그로 연결



전문 용어와 질문 표현을 연결하는 전처리이며, 모델 학습 자체를 뜻하지는 않습니다.

출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 7 / PDF 14쪽

한글 재구성 · 원문 번호 유지

그림 7 자동 태깅으로 질문과 전문 용어 연결

그림 7은 이 모듈의 작동을 보여 준다. 코퍼스에서 태그를 정의하고, 질문의 태그를 그 쪽 언어로 옮기는 두 방향의 흐름이 한 그림에 담긴다.

다층·다중 해상도 검색

용어의 벽까지 허물었으면 지식베이스를 제대로 훑을 차례다. L1은 L0에서 만든 다층 이중 그래프 위에서 다층·다중 해상도 검색을 수행한다. 그래프의 각 층은 추상화 수준과 해상도가 다른 지식을 담으므로, 질문을 문서 전체에 대응시킬 수도 있고 특정 청크에 대응시킬 수도 있다.

작동 원리는 심플하다. 질문과 그래프 노드를 고차원 벡터 임베딩으로 바꿔 유사도를 재고, 이 점수를 그래프 층 사이로 전파하고 집계한다. 정보 자원층의 노드 집합을 I , 코퍼스층을 C , 정제 지식층을 D 라 하고, 유사도 평가를 함수 g , 전파와 집계를 함수 f 로 나타내면 최종 청크 유사도 점수 S 는 다음과 같이 쓸 수 있다.

$$S = f(\{g(Q, n) \mid n \in I \cup C \cup D\})$$

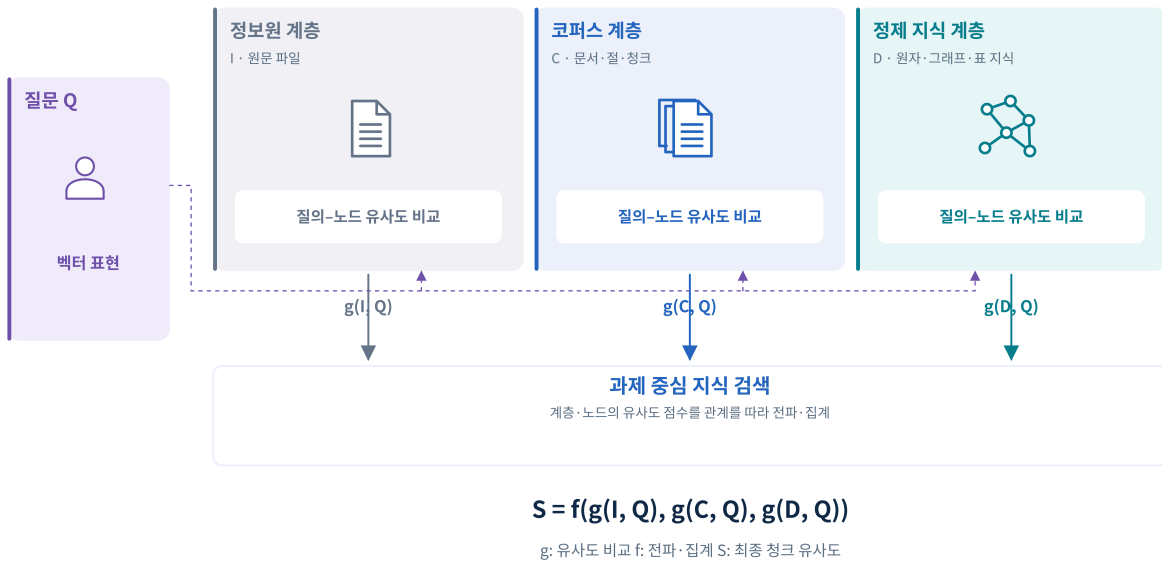
질문 Q 하나로 세 층의 점수를 모아 청크 점수를 매기는 구조다. 넓은 문맥과 세부 디테일을 동시에 반영하며, 과제 분해가 내놓은 하위 질문으로 검색을 반복해 다듬을 수도 있다.

PIKE-RAG / 한국어 그림 해설

08 / 18

다층·다중 해상도 지식 검색

문서 전체, 청크, 정제 지식을 함께 살펴 적절한 크기의 근거를 검색



출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 8 / PDF 15쪽

한글 재구성 · 원문 번호 유지

그림 8 다층·다중 해상도 지식 검색

그림 8은 다층 이중 그래프 위에서 층별로 유사도를 매기고 전파·집계하는 전경을 그린다. 하나의 질문이 여러 해상도의 지식을 동시에 조준하는 그림이다.

사실 너머의 질문으로

L1으로 사실 확인은 깔끔해졌다. 그러나 답 한 개가 코퍼스 한곳에 있지 않은 질문이 있다. 여러 문헌에서 근거를 모아 연결하고, 그 위에서 추론해야 하는 질문이다. 다음 장에서는 이를 위해 지식을 원자 단위로 쪼개는 L2 시스템을 만난다.

DAY

06

L2 — 지식을 원자로 쪼개다



“이 약과 저 약 중 어느 쪽이 이 환자에게 더 적합한가?” 같은 질문에는 답이 한 곳에 있지 않다. 여러 문헌을 넘나들며 근거를 모으고, 모은 근거 위에서 추론해야 한다. L2 시스템의 핵심 기능이 바로 이것이다. 관련 정보를 여러 소스에서 효율적으로 끌어 모아 복합 추론을 수행하는 능력이다.

L2의 목표와 구성

L2는 고급 지식 추출 모듈을 앞세워 필요한 정보를 빠짐없이 식별하고 추출한다. 여기에 과제 분해·조율 모듈을 더해 복잡한 과제를 감당할 만한 작은 하위 과제로 나눈다. 잘게 나눈 과제는 하나씩 해결하며 단계적으로 답에 다가선다.

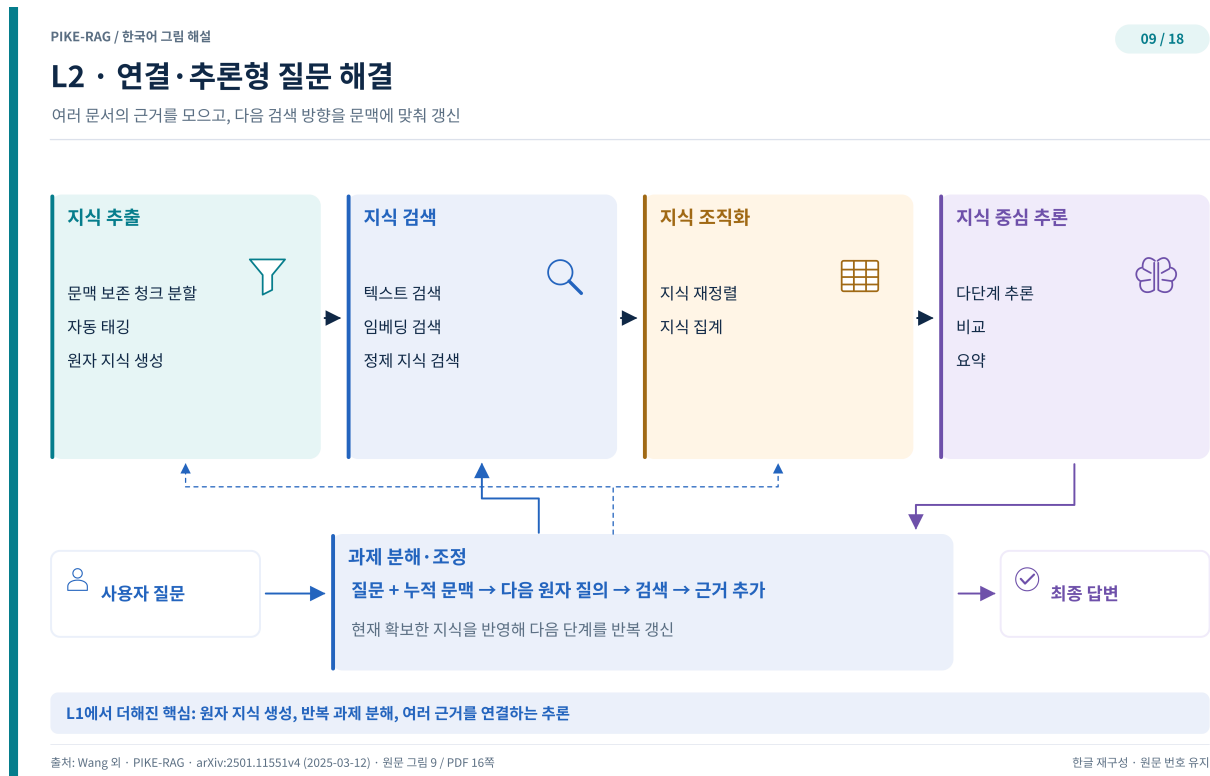


그림 9 L2 · 연결·추론형 질문 해결

그림 9는 L2 프레임워크의 전체 모습이다. 지식 추출 모듈에는 원자 지식 생성이, 지식 정리 모듈에는 지식 재순위화와 집계 하위 모듈이, 지식 중심 추론 모듈에는 다중 홑 추론·비교 추론·요약 하위 모듈이 자리 잡는다. 사실 확인을 넘어 연결과 추론을 감당하려면 각 모듈이 한 단계씩 진화해야 한다는 뜻이다.

체크 하나에 지식이 너무 많다

진화의 출발점은 분명한 문제의식이다. 체크 하나에는 여러 조각의 지식이 섞여 있다. 특정 과제를 풀 때 필요한 정보는 그 지식 전체에서 일부에 불과하다. 그런데도 전통적 정보 검색은 체크 통째로를 답의 그릇으로 취급하니, 정확히 필요한 정보를 효율적으로 꺼내기 어렵다.

한 가지 대안으로 체크에서 삼중항 지식 단위를 뽑아 지식그래프를 구축하는 연구들이 있었다. 그러나 지식그래프 구축은 비용이 크고, 체크에 담긴 지식이 그래프에 온전히 옮겨지지 않을 때가 많다. 문서에 박힌 지식을 더 충실하게 드러내려면 다른 길이 필요하다.

질문을 지식 색인으로 쓴다

PIKE-RAG가 택한 길은 지식 원자화(knowledge atomizing)다. 문서 청크 안의 원자 지식 조각들에 자동으로 꼬리표를 다는 작업으로, LLM의 문맥 이해와 생성 능력에 맡긴다. 여기서 다루는 청크는 원본 문서의 조각에 국한되지 않는다. 표와 그림, 영상을 위해 만든 설명 청크, 섹션이나 챕터, 문서 전체의 요약 청크도 모두 원자화 대상이다.

발상의 전환은 원자 지식을 어떻게 표현하느냐에서 나온다. 선언문이나 주어-관계-객체 삼중항 대신, 질문을 지식 색인으로 쓴다. 문서 청크를 LLM에 맥락으로 넣고, 그 청크가 답할 수 있는 질문을 최대한 많이 생성하도록 요청한다. 만들어진 원자 질문들은 해당 청크에 붙는 원자 질문 태그로 저장된다. 저장된 지식과 질의 사이의 간극을, 색인 자체를 질문 형태로 만들어 줄이는 것이다. 질문이 질문을 찾는 구조라 매칭이 자연스럽게 맞물린다.

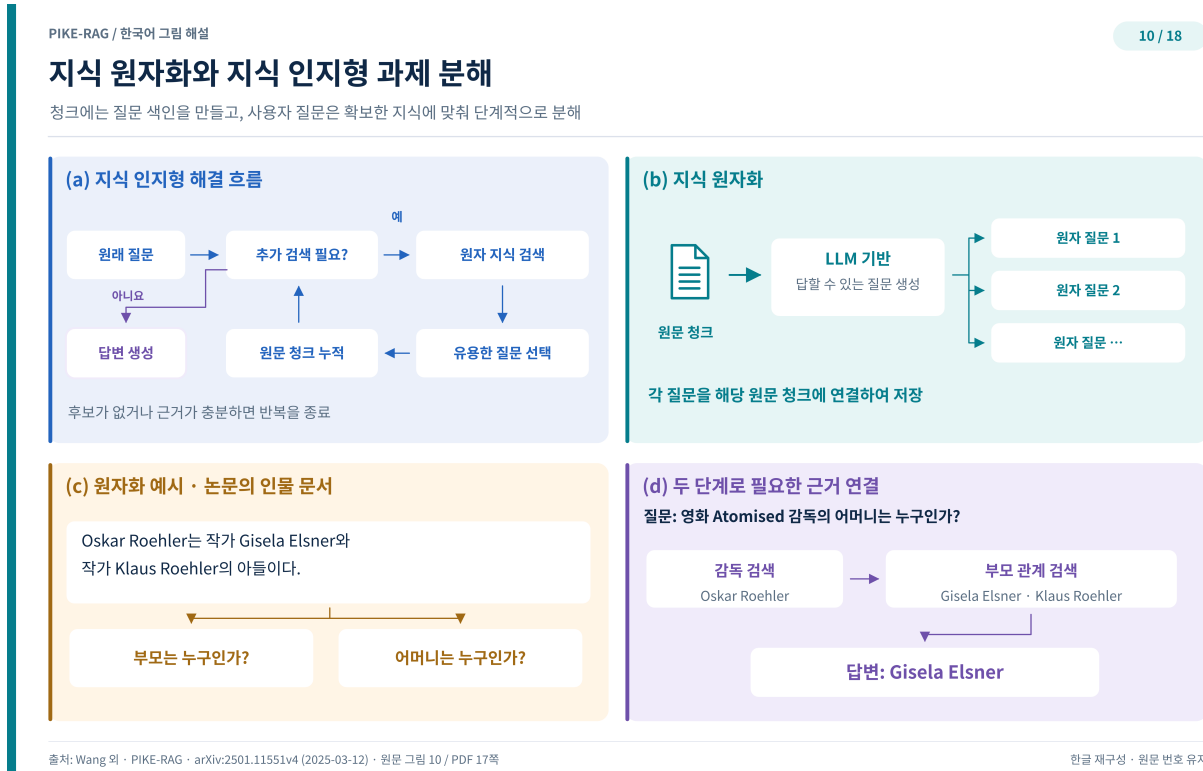


그림 10 지식 원자화와 지식 인지형 과제 분해

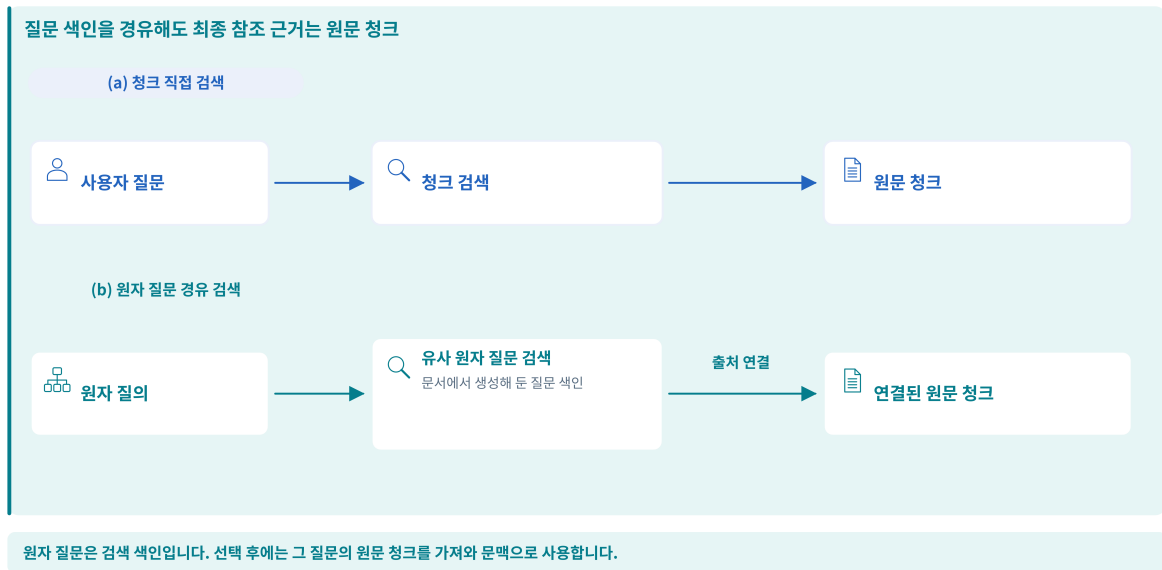
그림 10은 지식 원자화와 지식 인지형 과제 분해를 함께 보여 준다. 지식 인지형 과제 분해 워크플로, 원자화 절차, 원자화 예시, 그리고 둘을 결합한 RAG 사례까지 네 화면으로 구성된다. 원자 질문들이 청크 안의 지식을 여러 각도에서 조명하는 모습에 주목하자. 원자 지식과 원본 청크를 결합하면 계층형 지식베이스가 되고, 과제를 분해할 때마다 이 지식베이스가 어떤 지식이 있는지 알려 준다. 분해가 지식을 보고 이뤄지는 셈이다.

두 갈래 검색 경로

계층형 원자 지식베이스는 입자도가 다른 질의를 함께 받아 준다. 검색 경로가 두 개 열려 있기 때문이다. 첫째 경로는 익숙한 방식으로 질문이 참조 청크를 곧장 검색한다. 둘째 경로는 질문이 먼저 원자 질문 노드를 찾아내고, 그 원자 질문에 연결된 참조 청크로 이어진다.

원자 지식베이스의 두 검색 경로

청크를 직접 찾는 경로와 원자 질문을 거쳐 청크를 찾는 경로



출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 11 / PDF 18쪽

한글 재구성 · 원문 번호 유지

그림 11 원자 지식베이스의 두 검색 경로

그림 11은 이 두 경로를 나란히 그린다. 큰 질문은 청크 직접 검색으로, 잘게 쪼개진 질문은 원자 질문을 거쳐 청크로 향한다. 같은 지식베이스가 질문의 입자도에 맞춰 길을 바꿔 주는 것이다.

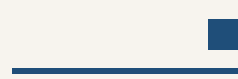
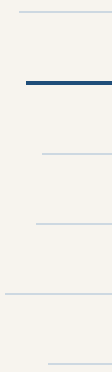
색인이 생겼다면 이제 분해다

지식을 원자로 쪼개고 질문 형태로 색인해 두니, 지식베이스가 스스로 “나에게는 이런 질문에 답할 지식이 있다”고 말할 수 있게 됐다. 그렇다면 과제를 분해하는 순간에 이 말을 들어 보면 어떨까. 어떤 지식이 있는지에 따라 분해 전략 자체가 달라진다. 다음 장에서는 지식베이스를 들여다보며 과제를 분해하는 알고리즘을 따라 간다.

DAY

07

L2 — 아는 만큼 나눈다



하나의 과제에 여러 분해 전략이 통할 때가 있다. 2장에서 만난 FDA 질문 Q2를 생각해 보자. 교환가능한 바이오시밀러 제품의 총수를 묻는 질문이다. 만약 지식베이스에 교환가능 제품 목록이 정리돼 있다면, 그 목록을 찾아 세기만 하는 두 단계 분석 추론으로 충분하다. 그런데 지식베이스에 바이오시밀러 제품의 일반 목록만 있고 속성이 여러 문서에 흩어져 있다면 이야기가 달라진다. 제품 목록을 찾고, 각 제품이 교환가능한지 판정하고, 교환가능한 제품 수를 세는 세 단계 분해가 필요하다. 어떤 분해가 최선인지를 가르는 결정적 기준은 딱 하나다. 전문 지식베이스 안에 무엇이 들어 있는지 아는 일이다.

지식이 분해를 가르친다

이 통찰에서 PIKE-RAG는 지식 인지형 과제 분해 워크플로를 설계했다. 분해기가 지식베이스에 무엇이 있는지 확인하면서 다음에 던질 질문을 고르는 방식이다. 원자 질문 색인이 있으니 지식베이스는 분해기에게 실제로 대답해 줄 수 있다. 알고리즘의 흐름을 따라 가 보자.

먼저 참조 맥락을 빈 집합으로 초기화한다. 원래 질문을 q 라 하자. 이후 반복이 최대 N 회 돌아가는데, t 번째 반복을 살펴보면 이렇다. LLM이 원래 질문과 지금까지 모은 맥락을 받아 과제 완수에 도움이 될 원자 질문 후보 여럿을 생성한다. 이때 이미 모은 참조 청크를 맥락으로 함께 주는 이유가 있다. 이미 아는 지식을 다시 묻는 후보가 나오지 않도록 막기 위해서다.

생성된 각 후보는 원자 질의로 쓰여 지식베이스에 관련 지식이 있는지 확인하는 데 들어간다. 원자 질문 후보마다 지식베이스에서 관련 원자 질문과 그 원천 청크를 상위 K 개 검색한다. 이때 검색 조건은 후보와 원자 질문의 유사도가 임계값 이상이라는 것이다.

$$\{(q_{ij}^t, c_{ij}^t) \in \mathcal{KB} \mid \text{sim}(q_{ij}^t, \hat{q}_i^t) \geq \delta\}$$

기호를 정리하자. q 는 원래 질문이고, t 는 반복 차수를 세는 지표다. K 는 후보 하나당 끌어 오는 원자 질문 개수이고, δ 는 검색에 통과할 최소 유사도다. 유사도 함수 sim 은 어떤 점수 척도도 허용하지만, 논문의 실험에서는 임베딩의 코사인 유사도를 썼다. 즉 임베딩 공간에서 후보와 거리가 δ 이상 가까운 원자 질문과 원천 청크 쌍만 후보군에 남는다.

검색된 원자 질문 목록을 손에 쥔 LLM은 원래 질문과 누적 맥락을 함께 보고 그중 가장 유용한 원자 질문 하나를 고른다. 선택된 질문에 해당하는 참조 청크를 꺼내 참조 맥락에 누적한다. 이 맥락이 다음 차수 분해의 입력이 된다. 반복이 끝나면 누적된 맥락 전체를 근거로 최종 답을 생성한다.

반복을 멈추는 네 가지 신호

분해가 항상 N 회까지 돌 필요는 없다. 다음 네 가지 조건 중 하나라도 해당하면 반복은 조기에 끝난다. 질 좋은 질문 후보가 더 나오지 않을 때, 유사도가 높은 원자 후보가 검색되지 않을 때, 마땅한 원자 지식을 고를 수 없을 때, 그리고 LLM이 지금 모은 지식으로 과제를 완수하기에 충분하다고 판단할 때다. 마지막 조건이 특히 실용적이다. 지식이 모였는데도 정해진 횟수를 채우려고 검색을 반복하는 것은 계산 비용만 늘리니, 충분함을 스스로 선언하는 순간 답 생성으로 넘어간다.

다음 과제는 분해기를 가르치는 것

여기까지의 흐름을 요약하면 이렇다. 지식베이스가 품은 지식을 원자 질문으로 색인해 두고, 분해기는 그 색인을 들여다보며 다음 질문을 정하고, 모은 근거 위에서 답을 낳는다. 다만 이 알고리즘은 매번 LLM의 판단에 의존한다. 과제 분해기 자체를 별도로 학습해 판단을 안정화할 수는 없을까.

알고리즘을 실제로 돌릴 때는 반복을 최대 5회, 최종 근거를 5청크로 제한한다. 이 설정이 실전에서 어떻게 작동하는지는 평가 장의 실험 설정에서 다시 만난다.

다음 장에서는 분해기를 학습하는 방법을 다룬다.

DAY

08

L2 — 분해기를 가르치다



지금까지의 과제 분해는 예시 프롬프트에 기대는 즉흥 연기에 가까웠다. 라운드마다 원자 질문 후보 여럿을 제안받고, 검색 결과를 살피고, 다음 질문을 고르는 데 각각 언어 모델 호출이 들었다. PIKE-RAG는 이 절차를 훈련 대상으로 바꾸는 길을 연다. 분해기를 가르치면 매 라운드의 호출 여럿이 제안자 호출 한 번으로 줄고, 분야 고유의 근거가 분해 규칙 자체에 스며든다.

■ 학습 가능한 구성요소

지식 인지형 과제 분해기(knowledge-aware decomposer)는 학습 가능한 구성요소다. 훈련된 제안자(trained proposer)는 추론 시점에 원자 질문을 곧장 제안하므로, 알고리즘에서 제안-검색-선택에 해당하는 여러 줄이 호출 한 번으로 대체된다. 원문은 이 교체가 추론 시간과 계산 비용을 함께 줄인다고 적는다. 절감이 전부는 아니다. 데이터 수집 과정에서 문맥 표본을 뽑고 다양한 상호작용 궤적을 만들어 두면, 분해기가 분야 고유의 근거를 과제 분해와 답 찾기에 반영하도록 가르칠 수 있다.

■ 점수 사전과 방문 사전

데이터 수집 절차는 이중 사전 관리에서 출발한다. 점수 사전(score dictionary)은 후보 청크마다 점수를 기록하고, 방문 사전(visit dictionary)은 청크별 방문 빈도를 센다. 초기화 단계에서 모든 점수는 0으로, 모든 방문 수는 1로 둔다. 수집용 파라미터는 실제 실행보다 넓게 잡는다. 유사도 임계값은 실제 실행의 δ 보다 낮은 δ' 를 쓰고, 후보 청크 개수는 실제 실행의 K 보다 큰 K' 로 뽑는다. 그물을 넉넉히 쳐야 답의 근거가 될 청크를 놓치지 않는다. 상위 K 안에 들지 못한 청크는 버리지 않고 점수 사전에 옮겨 관련도에 따라 점수를 갱신한다.

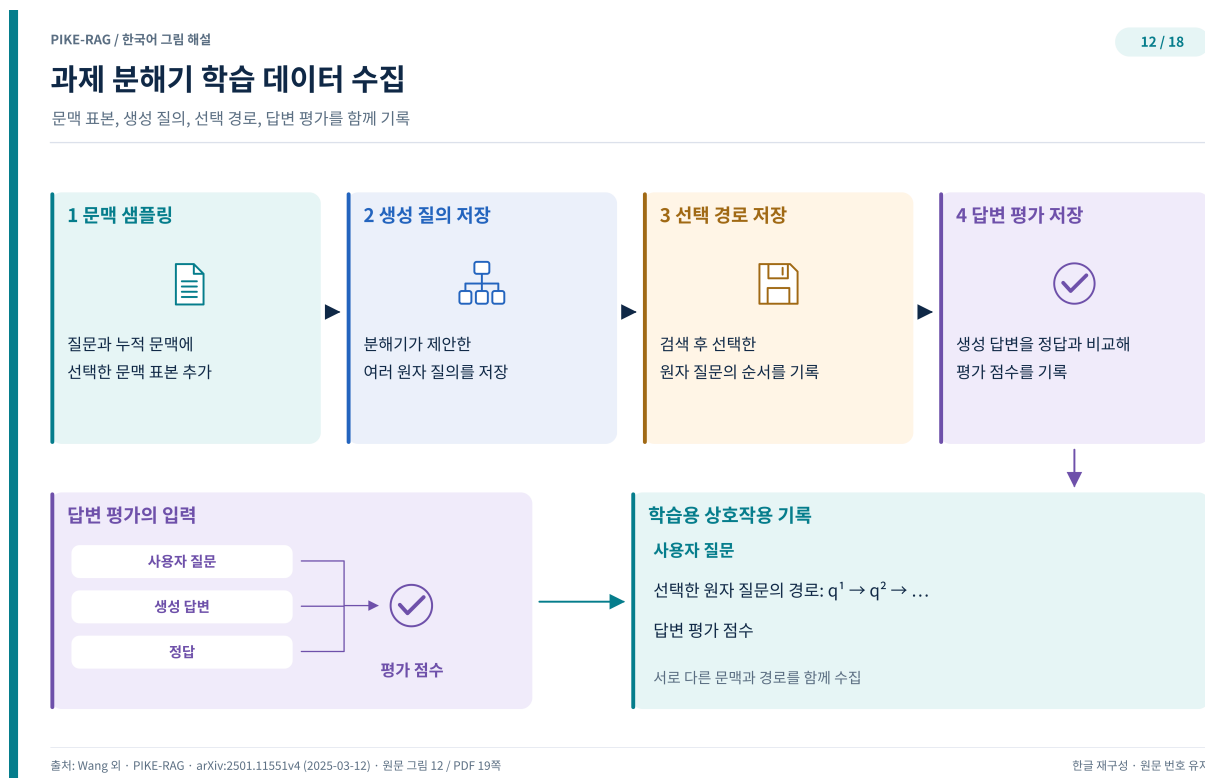


그림 12 과제 분해기 학습 데이터 수집

그림 12는 수집 절차를 네 구성요소로 요약한다. 문맥 표본 풀에서 청크를 뽑아 분해의 기준 문맥으로 삼고, 생성한 원자 질문 후보를 저장하고, 검색과 선택을 거쳐 뽑힌 후보를 추론 궤적에 붙이고, 마지막으로 답을 평가해 점수를 매긴다. 라운드가 거듭될 때마다 무엇을 보았고 무엇을 골랐으며 그 선택이 어떤 점수를 낳았는지가 차곡차곡 쌓인다.

이용과 탐색의 균형

문맥 표본 추출에는 상한 신뢰 한계(upper confidence bound, UCB) 알고리즘을 쓴다.

$$c_{sampled} = \arg \max_c \left(S(c) + \alpha \sqrt{\ln t / \mathcal{V}(c)} \right)$$

기호를 하나씩 풀면 다음과 같다. c 는 점수 사전에 등록된 후보 청크다. $S(c)$ 는 그 청크의 누적 점수이고 $\mathcal{V}(c)$ 는 방문 횟수다. t 는 지금까지의 반복 차수이며 α 는 탐색 세기를 조절하는 가중치다. 누적 점수가 높은 청크를 다시 고르는 이용(exploitation) 항과 덜 방문한 청크에 보너스를 주는 탐색(exploration) 항이 더해진다. 방문 횟수가 작을수록 제공된 항이 커지므로 아직 시험하지 않은 길이 기회를 얻는다. 이 장치는 검색기가 고른 청크로 당장의 성능을 챙기면서, 표본 추출로 불확실한 선택을 시험해 더 나은 대안을 발굴한다. 다양한 원자 질문 후보가 나오고 여러 추론 경로가 두루 탐색되는 것은 이 균형의 결과다.

궤적을 감독 신호로

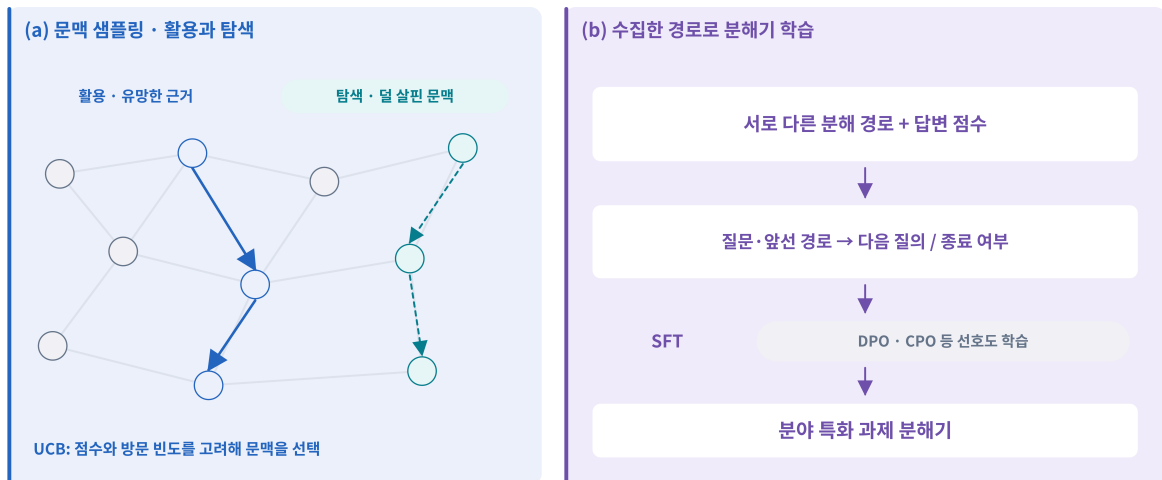
원자 제안(atomic proposal)과 상호작용 궤적과 답 점수를 기록해 분해기 학습에 쓴다. 분야마다 추론 경로가 다르므로 분야별로 궤적을 모으고, 답 점수를 감독 신호(supervised signal)로 삼아 분해기를 훈련한다.

PIKE-RAG / 한국어 그림 해설

13 / 18

문맥 탐색과 분야 맞춤 분해기 학습

이미 유망한 문맥을 활용하면서, 다른 문맥과 분해 경로도 탐색



그림의 선호도 학습은 확장 선택지입니다. 이 버전 6.4절에 제시된 분해기 학습 실험은 SFT입니다.

출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 13 / PDF 19쪽

한글 재구성 · 원문 번호 유지

그림 13 문맥 탐색과 분야 맞춤 분해기 학습

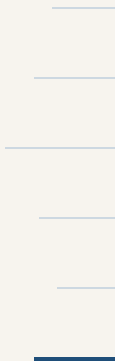
그림 13의 왼쪽은 문맥 표본 추출이 어떻게 다양한 원자 질문 후보를 낚는지를 보여주고, 오른쪽은 모은 데이터로 분해기를 가르치는 과정을 그린다. 답 점수라는 선호 신호가 분해 규칙에 스며들면 분해기가 해당 분야의 요구에 맞게 적응한다. 훈련 방법으로는 지도 미세조정(supervised fine-tuning, SFT)과 직접 정책 최적화(direct policy optimization, DPO)를 기존 언어 모델 위에 얹을 수 있다. 방법론 시점의 원문은 이를 후속 과제로 남겨 두었지만, 같은 논문은 뒤의 평가 절에서 도메인 정렬 제안자 실험으로 실제 수행한다. 그 결과는 법률 벤치마크장에서 만난다.

분해기가 스스로 질문을 던지는 단계까지 왔다. 그러나 어떤 질문은 아무리 잘 쪼개도 답이 자료 어디에도 없다. 다음 장에서는 답을 찾는 질문에서 답을 만들어 내는 질문으로 넘어간다.

DAY

09

L3 — 미래를 묻는 질문



“내년까지 승인될 신약은 몇 건일까.” 이런 질문의 답은 아무리 좋은 검색기로도 문서에서 튀어나오지 않는다. 자료에는 과거의 승인 기록만 있고 미래의 숫자는 없기 때문이다. L3 시스템은 여기서 출발한다. 답이 원문에 없는 예측 질문을 풀려면 지식을 모으는 일보다 지식을 구조화하는 일이 먼저다. 3장에서 예고한 지식 구조화와 지식 유도 서브모듈이 이 단계에서 실제로 움직인다.

지식 구조화와 지식 유도

L3는 지식 기반 예측 능력에 무게를 둔다. 검색된 지식을 모으고 조직한 다음, 그 위에 예측 근거(forecasting rationale)를 세운다. 과제 분해와 조율 모듈이 조직된 지식 위에서 예측 근거를 만드는 역할을 맡는다. 지식 조직(knowledge organization) 모듈에는 두 서브모듈이 새로 붙는다. 지식 구조화(knowledge structuring)는 원시 검색 지식을 구조화된 일관된 형태로 바꾸어 뒤이은 추론과 예측에 쓰기 좋게 만들고, 지식 유도(knowledge induction)는 구조화된 지식을 범주로 묶어 통계 분석과 예측의 밑천으로 삼는다.

원문이 드는 예시가 미국 식품의약국(FDA) 시나리오다. 약품 라벨, 임상시험, 신청 서류 같은 여러 원천의 데이터가 다층 지식베이스로 들어온다. 지식 구조화 서브모듈은 과제 분해 모듈의 지시에 따라 약품명과 승인일 같은 관련 지식을 수집하고 조직한다. 지식 유도 서브모듈은 그 지식을 승인일 같은 기준으로 범주화해 통계 분석과 예측을 돕는다.



그림 14 L3 · 자료 구조화에서 예측으로

그림 14는 L3 프레임워크의 전체 흐름을 그린다. 지식 조직 모듈 안의 두 사각형이 지식 구조화와 지식 유도를 가리키고, 지식 중심 추론 모듈 안의 사각형이 예측 서브모듈을 가리킨다. 원시 지식이 구조화되고 범주화된 다음에야 예측이 가능하다는 순서가 그림에 그대로 드러난다.

예측 서브모듈

언어 모델은 전문 분야의 추론 논리를 적용하는 데 한계가 있어 예측 과제에서 효과가 제한된다. 지식 중심 추론(knowledge-centric reasoning) 모듈은 이를 보완하려고 예측(forecasting) 서브모듈을 새로 둔다. 질문과 조직된 지식을 받아 연도별 승인 약품 총수 같은 통계에서 결과를 추론한다. 시스템은 과거 지식에 근거한 답변을 넘어 투영(projection)을 수행하므로 복잡하고 역동적인 질문에 더 견고한 답을 낸다. 봉지에 담긴 날짜별 영수증만 나열해서는 이번 달 지출을 알

수 없고 항목별로 묶어 봐야 추세가 보이는 것과 같다. 지식 유도가 묶음을 만들고 예측 서브모듈이 그 추세를 다음 달로 밀어 간다. 분해와 조율 모듈이 예측에 쓸 근거를 설계하고, 추론 모듈의 예측 서브모듈이 그 실행을 맡는다.

■ 답이 하나로 정해지지 않는 질문

예측 질문의 답은 유일하지 않다. EM이나 F1 같은 지표는 사실이고 확정적인 답을 전제하므로 예측 질문에는 어울리지 않으며, 추세 판단과 정성 분석이 더 적합하다. 평가 방식 자체도 단계와 함께 바뀌어야 한다.

지식을 배열해 미래를 읽는 단계까지 왔다. 다음 장에서는 배열된 지식으로 새로운 것을 만들어 내는 창의 질문을 만난다.

DAY

10

L4 — 관점을 늘리는 에이전트들



“이 지식으로 새로운 무엇을 만들 수 있을까.” 창의 질문은 사실 정보에 바탕을 두되 원리와 규칙을 제대로 이해해야 풀린다. 정해진 답을 찾는 질문과 결이 다르다. L4 시스템은 여러 에이전트를 엮어 다관점 사고(multi-perspective thinking)를 가능하게 하는 것으로 이 단계를 넘는다.

창의 질문의 세 난제

원문은 창의 질문을 다루는 데 세 가지 어려움을 꼽는다. 첫째는 검색된 지식에서 일관된 논리 근거를 뽑아내는 일이다. 근거 조각이 흩어져 있으면 창의적 제안도 공중에 뜬다. 둘째는 여러 영향 인자가 얽힌 복잡한 추론 과정을 밟아 가는 일이다. 셋째는 개방형 질문의 답을 평가하는 일이다. 앞 단계의 지표는 확정 답을 전제하므로, 관련성과 다양성과 포괄성과 독창성과 영감 같은 기준으로 채점 방식을 새로 설계해야 한다.

멀티 에이전트 플래닝

해법은 멀티 에이전트 플래닝(multi-agent planning)이다. 에이전트마다 고유한 통찰과 추론 전략을 담당하며 병렬로 움직인다. 서로 다른 사고 과정이 동시에 돌고 그 결과가 합쳐지므로, 다양한 추론 경로를 병렬로 처리하고 통합하는 일이 한번에 이루어진다. 이 구조는 얽힌 질문도 제대로 관리하고 응답할 수 있게 한다. 여러 관점을 시뮬레이션하므로 시스템은 정해진 해법을 출력하는 대신 혁신적인 아이디어를 낳는다. 여러 에이전트의 조율된 출력은 추론 과정을 풍부하게 하고 사용자에게 포괄적 관점을 건네며, 창의적 사고와 새로운 해법의 영감을 불어넣는다.

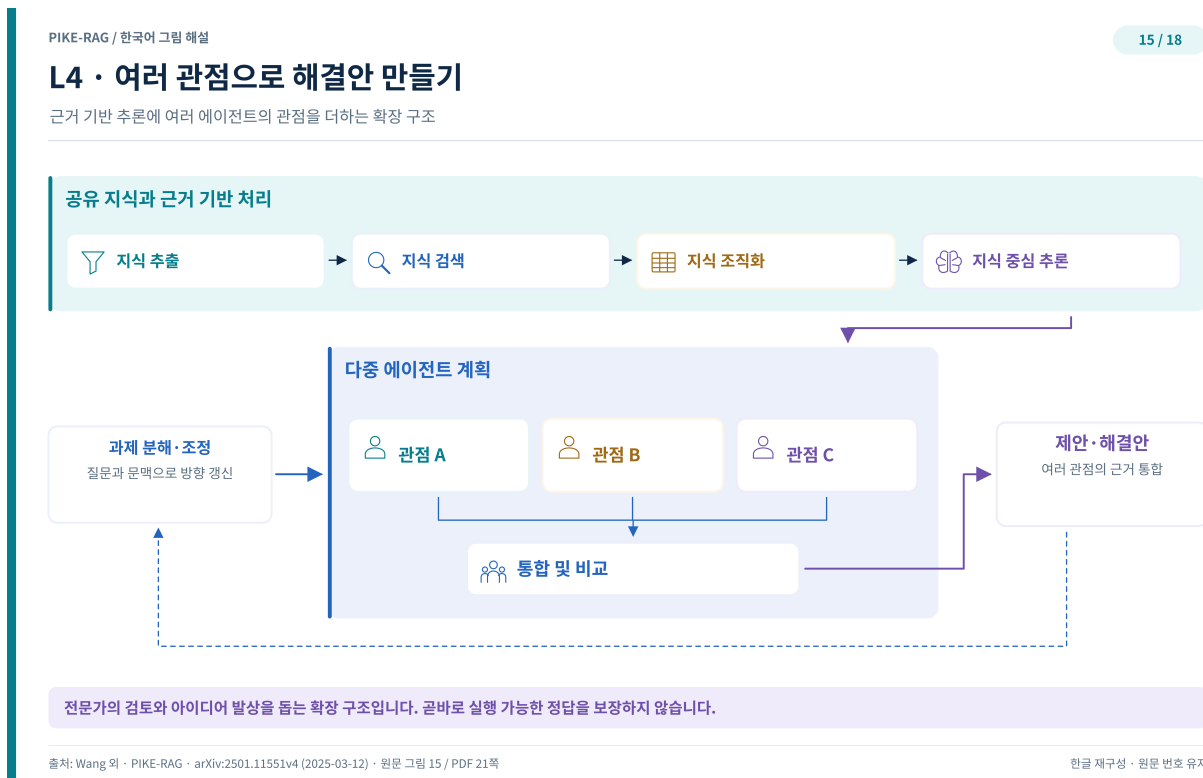


그림 15 L4 · 여러 관점으로 해결안 만들기

그림 15는 멀티 에이전트 플래닝 모듈이 추가된 L4 프레임워크를 보여준다. 여러 에이전트가 병렬로 추론 경로를 걷고 그 출력이 합쳐진다. 한 경로의 답을 고르는 구조가 아니라 경로 여럿을 합성하는 구조라는 점이 이전 단계 그림과 다르다. 회의실에서 여러 전문가가 각자 의견을 내놓으면 한 사람의 결론보다 넓은 선택지가 모이는 것과 같은 원리다.

| L1부터 L4까지, 하나의 프레임워크

다섯 단계를 돌아보자. L0은 지식베이스를 구축하고 L1은 사실 질문에 답하며 L2는 여러 근거를 잇고 L3는 지식을 구조화해 예측하며 L4는 창의적 해결안을 제안한다. 단계가 오를 때마다 시스템 전체를 갈아엎는 일은 없다. 일곱 모듈로 이루어진 하나의 프레임워크 안에서 필요한 모듈을 보강하고 서브모듈을 붙이는 방식으로 능력이 쌓인다. 이 계단식 설계 덕분에 실무자는 자신의 데이터와 질문 수준에 맞는 단계에서 출발해 한 단계씩 올라갈 수 있다.

이론의 계단을 모두 밟았다. 다음 장부터는 이 설계가 실제로 통하는지 벤치마크 수치로 확인한다.

DAY

11

벤치마크로 검증하다 — 오픈도메인



설계가 좋다는 말은 수치로 증명한다. PIKE-RAG는 오픈도메인 벤치마크 세 종과 도메인 벤치마크 두 종에서 성능을 재었고, 이 장에서는 그중 오픈도메인 결과를 읽는다. 실험 설정을 먼저 확인하고, 세 벤치마크의 성적표를 표로 옮긴 다음, 수치가 말하는 서사를 짚는다.

실험 설정

모든 방법이 같은 언어 모델 GPT-4(gpt-4-1106-preview)를 쓴다. 지식 원자화 단계만 temperature 0.7로 두어 원자 지식의 다양성과 안정성 사이 균형을 맞추고, 질의응답 단계는 모두 temperature 0으로 두어 일관된 응답을 얻는다. 임베딩은 text-embedding-ada-002 버전 2를 평탄 지식베이스와 계층 지식베이스 양쪽에 쓴다. 평탄 지식베이스 검색기는 최대 16개 청크를 가져오되 검색 점수 임계값을 0.2로 둔다. 계층 지식베이스 검색기는 더 엄격한 임계값 0.5로 최대 8개 청크를 가져오고, 원자 질문 하나당 추가로 4개 청크를 더 가져올 수 있다.

비교 대상은 여섯 계열이다. Zero-Shot CoT는 외부 문맥 없이 단계별 사고만으로 답해 언어 모델의 내재 지식을 쟀다. Naive RAG는 평탄 지식베이스에서 밀집 검색으로 문맥을 얻는다. Self-Ask는 후속 질문을 반복 생성해 과제를 쪼개되 외부 문맥 없이 내재 지식으로 답한다. Self-Ask에 검색을 얻은 변형과 계층 검색을 얻은 변형도 함께 뛰었다. GraphRAG는 지식 그래프를 만들어 지역 모드와 전역 모드로 돌린다. 우리 방법은 지식 인지형 과제 분해로 최대 5회까지 질문을 쪼개고 답의 근거를 가장 쓸모 있는 청크 다섯으로 제한한다.

지표는 다섯이다. EM(Exact Match)은 응답이 정답과 완전히 같은지를 보고, F1은 토큰 수준의 정밀도와 재현율의 조화 평균을 낸다. 여기에 GPT-4를 채점자로 세운 Acc(Accuracy)를 더한다. 채점자 모델은 질문과 정답 레이블을 놓고 응답의 옳음을 판정하는데, 표본을 사람이 직접 검사한 결과 인간 평가와 100% 일치했다. 정답 레이블이 여럿일 때는 EM·F1·정밀도·재현율은 가장 높은 점수를 취하고 Acc는 모든 정답 레이블을 함께 준다.

벤치마크 세 종

HotpotQA는 2홉 질문이 주를 이루는 가장 널리 알려진 멀티홉 벤치마크다. 각 질문에 위키피디아 문단 10개가 붙고 그중 일부만 답의 근거를 담으며 나머지는 방해 역할을 한다. 질문 유형은 두 엔티티를 건주는 비교형과 중개 엔티티를 거쳐 답에 이르는 브리지형으로 나뉜다.

2WikiMultiHopQA는 HotpotQA의 비교형을 남겨 두되 엔티티 속성 추론의 추론형, 엔티티 위치 찾기의 구성형, 그리고 둘을 섞은 브리지 비교형까지 네 유형으로 폭을 넓혔다. 2홉에서 4홉 사이의 질문마다 문단 10개가 따라온다.

MuSiQue는 지름길로 풀리는 질문을 건어내는 데 가장 공격적이다. 제대로 된 연결 추론 없이 정답에 닿을 수 있는 지름길을 엄격한 필터로 차단하고, 2홉에서 4홉 사이의 각 질문에 문단 20개를 붙여 관련 정보와 무관한 정보를 뒤섞는다.

세 데이터셋의 dev 세트에서 질문 유형과 홉 수를 가리지 않고 각각 500문항을 무작위로 뽑았고, 표본의 문단을 벤치마크 별로 한 지식베이스에 합쳐 검색 난도를 높였다. 우리 방법은 문제를 푸는 동안 유형 정보와 홉 수 정보를 쓰지 않는다.

전체 성적

세 벤치마크의 Acc 성적을 한 표에 모았다. 원문 표 4부터 6까지의 열을 벤치마크 축으로 접었다.

표 11-1 세 벤치마크 Acc 비교

방법	HotpotQA	2WikiMultiHopQA	MuSiQue
Zero-Shot CoT	53.60	43.87	23.47
Naive RAG w/ R	82.60	62.80	44.40
Self-Ask w/ H-R	82.20	80.00	54.00
GraphRAG Local	89.00	71.20	49.80
Ours	87.60	82.00	59.60

표 11-2 우리 방법의 세부 지표

벤치마크	EM	F1	Acc	정밀도/재현율
HotpotQA	61.20	76.26	87.60	78.10/78.95
2WikiMultiHopQA	66.80	75.19	82.00	74.04/78.87
MuSiQue	46.40	56.62	59.60	57.45/59.53

벤치마크 설계상 난이도가 고르게 나뉘었다고 보면, 최고 성적을 난이도의 대리 지표로 삼을 때 쉬운 순서는 HotpotQA, 2WikiMultiHopQA, MuSiQue다. HotpotQA에서는 GraphRAG Local과 우리 방법이 근소한 차이로 맞붙지만, 난이도가 오르는 2WikiMultiHopQA와 MuSiQue에서는 우리 방법이 다른 방법을 앞선다.

검색을 엮는 것만으로도 Acc가 크게 오른다. Zero-Shot CoT와 Naive RAG를 비교하면 개선 폭이 벤치마크마다 다른데, 원문은 약 10%에서 약 29%까지라 표기한다. 다만 MuSiQue는 표 11-1로 계산하면 23.47에서 44.40으로 약 21포인트 오른다. 쉬운 벤치마크는 순진한 검색만으로도 멀티홉 질문을 넘보지만, 어려운 벤치마크에서는 검색만으로 얻는 개선 폭이 줄고 언어 모델 자체 추론의 한계가 드러난다. Self-Ask가 분해를 없으면 특히 어려운 벤치마크에서 Acc가 유의미하게 오르고, 검색과 분해를 함께 쓰면 2WikiMultiHopQA와 MuSiQue에서 한 가지만 쓸 때보다 좋다.

계층 원자 지식베이스의 이득은 분해와 결합할 때 나타난다. Naive RAG에 계층 검색을 엮어도 평탄 지식베이스를 쓸 때와 성적이 크게 달라지지 않는다. 원래의 멀티홉 질문과 관련 문맥의 원자 질문 사이 임베딩 거리가 그 이유다. 그러나 Self-Ask처럼 분해 기제와 결합하면 이야기가 달라진다. 검색만 엮은 Self-Ask보다 계층 검색을 엮은 Self-Ask가 MuSiQue에서 더 높은 Acc를 기록했고, 추론이 복잡한 벤치마크에서 계층 지식베이스가 분해 기제의 효과를 키운다는 것을 보여준다.

GraphRAG의 성적표에는 아이러니가 적혀 있다. HotpotQA에서 GraphRAG Local은 EM 0.00인데 Acc는 89.00으로 전체 1위다. 출력물을 들여다보면 이유가 밝혀진다. GraphRAG는 질문을 그대로 반복하고 그래프 구조의 메타정보를 답에 남기는 습관이 있는데, QA 프롬프트를 다듬어도 이 습관이 사라지지 않았다. 원문 표 7의 사례가 이를 보여준다. Alsa Mall과 Spencer Plaza가 어느 나라에 있는지 묻는 질문에 GraphRAG Local은 인도의 첸나이라고 옳게 답하면서도, 답 뒤에 커뮤니티와 엔터티와 관계의 번호를 알리는 메타정보 나열을 그대로 붙인다. 이 잔여물이 응답을 길게 부풀려 EM과 정밀도는 바닥을 치고 Acc와 재현율은 치솟는다.

오픈도메인에서 검증을 마쳤다. 다음 장에서는 같은 질문을 법률 벤치마크로 옮겨 전문 분야에서의 성적을 읽는다.

DAY

12

법률 벤치마크와 분해기 파인튜닝



오픈 도메인에서 좋은 성적을 냈다고 전문 분야에서도 그럴 것이라고 단정할 수는 없다. 법률처럼 조문을 정확히 가리켜야 하는 영역에서는 검색이 조금만 어긋나도 답이 무너진다. 이 장은 두 법률 벤치마크의 수치를 읽고 같은 자료로 지표를 바꾸면 갭은 벌어지지만 등수는 지켜지는 현상을 파헤친 다음, 작은 언어 모델을 파인튜닝해 질문 분해기를 만들면 얼마나 이득인지 살펴본다.

두 법률 벤치마크

논문이 고른 첫 벤치마크는 중국 법률용 LawBench다. 법률 개체명 인식부터 독해, 범죄 금액 계산, 법률 상담까지 스무 개 과제로 이루어져 있으므로, 검색 증강과 상관이 없는 독해 같은 과제를 걸어내고 여섯 과제를 골랐다. 각 과제마다 문항 500개를 사용했다.

- 조문 암기 1-1: 법 조항 내용을 그대로 생성하며 지표는 F1이다.
- 법률 지식 QA 1-2: 단일 선택 문제이며 지표는 EM이다.
- 조문 예측(사실 기반) 3-1: 사실 관계와 혐의에서 해당 조문을 고른다. 복수 선택이며 지표는 EM이다.
- 조문 예측(시나리오 기반) 3-2: 시나리오에 맞는 법적 근거 조항을 생성하며 지표는 F1이다.
- 사건 분석 3-6: 사건에 비추어 유효한 의결을 판별하는 단일 선택이며 지표는 EM이다.
- 법률 상담 3-8: 상담 질문에 법적 근거를 대는 답을 생성하며 지표는 F1이다.

두 번째 벤치마크는 Open Australian Legal QA다. 호주 법률 코퍼스에서 GPT-4가 만들어 낸 2,124개의 질문과 답으로 이루어져 있으며, 모든 질문이 생성형이다.

지표를 바꾸면 갭은 벌어지고 등수는 지켜진다

논문은 Zero-Shot CoT와 GraphRAG Local, 그리고 자신들의 방법을 나란히 놓고 비교했다. GPT-4가 답의 의미를 읽고 정확도(Acc)를 매긴 결과가 표 12-1이다.

표 12-1 법률 벤치마크 정확도 비교(단위는 Acc)

과제	Zero-Shot CoT	GraphRAG Local	Ours (N=5)
조문 암기 1-1	1.23	16.60	90.12
법률 지식 QA 1-2	54.00	63.40	70.60
조문 예측(사실 기반) 3-1	49.90	75.40	88.82
조문 예측(시나리오 기반) 3-2	15.83	27.60	67.54
사건 분석 3-6	51.12	57.00	62.73
법률 상담 3-8	49.70	58.80	61.72
Open Australian Legal QA	16.48	88.27	98.59

이 표만 보면 격차가 터무니없어 보이는데, 답은 같은 실험의 F1 점수에 있다. 토큰 단위로 재는 F1로 보면 조문 암기에서 세 방법의 점수가 각각 21.31, 23.27, 78.58로 잡히고, 시나리오 기반 조문 예측에서는 27.51, 25.98, 46.05가 된다. 격차는 줄지만 등수는 보존된다. 이 방법의 답은 황금 답과 의미가 같으면서 어순이나 어휘가 다른 경우가 많고, 토큰을 겹쳐 재는 F1은 그런 의미적 동치를 점수로 못 바꾸지만 GPT-4가 의미를 비교하는 Acc는 온전히 인정한다. 생성형 과제에서 이 방법의 Acc가 유독 크게 오른 것은 이 때문이다.

반대로 GraphRAG Local의 Acc는 조문 암기에서 F1보다 내려간다. 특정 조문을 찾아야 하는 과제에서 틀린 조문을 인용하면 답이 틀리지만, 이 방식은 법률 정보를 반복해서 늘어놓는 경향이 있어 토큰 단위 재기에는 유리했다. 조항 이름과 앞 머리 구문만 바꾸어도 F1이 올라갈 수 있다는 뜻이다.

선택형 과제인 1-2, 3-1, 3-6에서는 두 지표가 예상대로 거의 겹친다. 예외인 3-1에서는 수사를 아랍 숫자로 쓸지 한자 숫자로 쓸지에 따라 GPT-4 판정이 갈린다. 법률 상담 3-8에서 두 방법 모두 Acc가 크게 오른 데에는 또 다른 사정이 있다. 질문과 황금 답을 상담 웹사이트에서 가져오는 바람에 품질이 고르지 못한 것이다. 친생 부모가 이혼하고 새 가정을 꾸린 뒤

자녀의 부양 의무를 묻는 문항의 황금 답에는, 미성년 자녀를 부양할 부모의 의무를 규정한 제1067조라는 무관한 조문이 근거로 붙어 있다. 평가셋 자체가 흔들리면 지표도 흔들린다는 교훈이 여기서 나온다.

작은 분해기를 가르친다

오픈 도메인 벤치마크에서 방법 전반이 낮은 점수를 낸 MuSiQue를 골라 질문 분해기 파인튜닝 실험을 설계했다. 도메인 정렬 원자 제안자가 그 대상이다. 데이터셋에서 문항 1,000개를 무작위로 뽑고 80 대 20으로 나누었다. 80퍼센트는 분해 궤적을 모으는 학습용이고 나머지는 평가용이다. 학습용 문항마다 후속 질문을 단계적으로 만들어 정답에 이르는 분해 궤적을 확보하되 문항당 최대 열 라운드를 두었다. 문맥 샘플링 기법을 더하자 궤적 확보에 성공한 비율이 58퍼센트에서 84퍼센트로 올라갔다.

학습은 지도 미세조정(SFT)으로 진행했다. 성공한 궤적 하나를 원래 질문과 원자 질문·답의 쌍들로 이루어진 학습 데이터로 바꾸는데, 원자 질문이 t개인 궤적은 학습 사례 t+1개를 낳는다. 학습률은 1.5e-5로 두고 LoRA 설정(rank 16, alpha 64)으로 매개변수 효율 미세조정을 적용했다. 실험은 서로 다른 무작위 시드로 세 번 돌려 평균을 냈고, 학습에는 NVIDIA A100-80G 한 장을 썼다.

분해기 베이스 모델로는 크기가 다른 오픈소스 셋을 골랐다. Llama-3.1-8B, Qwen2.5-14B, phi-4-14B가 분해를 맡고, 최종 답 생성은 GPT-4o와 Llama-3.1-70B-Instruct가 나누어 수행했다. 결과가 표 12-2다.

표 12-2 도메인 정렬 원자 제안자 파인튜닝 효과(MuSiQue, 단위는 Acc)

분해기 베이스 모델	GPT-4o	GPT-4o+FT	Llama-3.1-70B	Llama-3.1-70B+FT
Llama-3.1-8B	47.83	62.14	48.37	58.70
Qwen2.5-14B	56.52	63.95	57.61	63.04
phi-4-14B	60.33	65.76	58.70	62.50

표를 가로질러 읽으면 파인튜닝의 효과가 드러난다. FT가 붙은 열이 항상 붙지 않은 열을 앞선다. 세로로 읽으면 또 하나의 사실이 보인다. Llama-3.1-8B를 가르친 뒤 GPT-4o와 짝지은 62.14는, 학습하지 않은 여덟 배가량 큰 Llama-3.1-70B-Instruct가 분해를 맡은 48.37을 훌쩍 넘는다. 최종 답 모델을 크게 교체하는 것보다 작은 분해기를 도메인 궤적으로 가르치는 편이 이득이라는 결론이며, 분해 품질이 최종 답의 상한선을 정한다는 사실을 수치가 뒷받침한다.

법률 벤치마크와 분해기 학습까지 읽었다면 마지막 과제가 남았다. 다음 장에서는 같은 파이프라인이 실제 질문에서 실패를 어떻게 되돌리는지 세 사례로 짚어본다.

DAY

13

세 가지 실패 사례와 해법



수치 표는 격차가 얼마인지는 알려주지만 격차가 왜 생기는지는 말해주지 않는다. 논문은 평가 벤치마크에서 가져온 실제 사례 세 개를 펼쳐 놓고, 단일 경로 분해가 어디서 길을 잃는지 그리고 이 방법의 파이프라인이 어떻게 그 실패를 되돌리는지 대조한다. 세 사례는 각각 따로 있는 문제처럼 보이지만, 뒤집으면 하나의 원칙이다. 질문의 의도를 이르게 확정하지 말고, 원문의 표현과 질문의 표현 사이 간극을 좁히고, 원문 문맥을 끝까지 놓지 말라는 것이다.

사례 A. 비슷한 제목을 잘못 고르는 문제

첫 사례는 언어 모델의 선입견이 질문 자체를 교정해 버리는 상황이다. 원래 질문은 덜 유명한 영화 What Women Love를 묻는다. 그런데 세상에 훨씬 유명한 영화 What Women Want가 있으니, 모델은 원래 질문을 유명한 쪽으로 바꾸어 읽어 버린다. Self-Ask처럼 후속 질문을 하나만 만드는 방식은 이 잘못된 대상에 대한 후속 질문을 생성하고, 그로부터 거짓 중간 답을 얻어 최종 답까지 틀리게 도달한다. 재미있는 점은 목표 청크가 임베딩 유사도 덕에 이미 검색되었다는 사실이다. 검색은 성공했는데 질문 해석이 틀려 망한 경우다.

PIKE-RAG / 한국어 그림 해설

16 / 18

사례 A · 비슷한 제목을 잘못 고르는 문제

원래 질문의 대상을 유명한 다른 대상으로 바꾸면, 올바른 검색도 무용해질 수 있음

원래 질문

What Women Love와 Ramudu Kadu Krishnudu 중 먼저 나온 영화는?

Self-Ask · 논문의 오류 사례

질문의 제목을 바꿈

What Women Love → What Women Want

잘못 고른 대상의 연도를 사용

중간 답변: 2011 (원문에 보고된 오류)

❌ 잘못된 결론

최종 답변: Ramudu Kadu Krishnudu

PIKE-RAG · 논문의 해결 경로

여러 원자 질의 제안

정확한 제목 · 개봉 시점 · 비슷한 제목의 후보

원래 질문의 대상을 근거와 대조

What Women Love에 해당하는 원자 질문 선택

✅ 원래 대상의 문서 근거 선택

1920년의 What Women Love 문서

특정 실험 사례의 비교입니다. 모든 질문에서 각 방법이 동일하게 성공하거나 실패한다는 뜻은 아닙니다.

출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 16 / PDF 30쪽

한글 재구성 · 원문 번호 유지

그림 16 사례 A · 비슷한 제목을 잘못 고르는 문제

그림 16의 원편에서 Self-Ask가 유명한 제목 쪽으로 후속 질문을 하나만 던지고 잘못된 결론에 도착하는 경로를 확인할 수 있다. 오른쪽의 이 방법은 What Women Love와 What Women Want 양쪽에 관한 원자 질문 여러 개를 제안한다. 두 영화가 모두 존재하고 관련 원자 질문이 검색되었으므로, 이 방법은 질문의 진짜 의도를 판단하는 일을 뒤로 미룰 수 있다. 의도 판정을 처음에 확정하지 않고 원자 질문 선택 단계로 옮기는 것이 핵심이다. 후보가 여럿일 때 비로소 어느 쪽이 질문자의 뜻인지 근거를 놓고 고를 수 있다.

사례 B. 질문과 원문의 표현 차이

두 번째 사례는 질문이 옳아도 검색이 실패하는 상황이다. Self-Ask 같은 단일 경로 방식은 Oskar Roehler의 어머니가 누구인지 곧장 묻는 후속 질문을 만든다. 뜻은 정확하다. 그러나 지식베이스는 가족 관계를 A가 B와 C의 아들이더라는 스키마로 서술하고 있으므로, 어머니라는 표현으로는 해당 청크가 끌려 나오지 않는다. 계층적 검색을 Self-Ask에 얹어도 이 간극을 메우지 못했다. 질문의 의미가 맞아도 원문의 문장 구조와 어긋나면 검색은 멈춘다.

사례 B · 질문과 원문의 표현 차이

같은 관계도 질문과 문서에서는 서로 다른 문장 형태로 표현될 수 있음

원문에 기록된 관계

Oskar Roehler는 Gisela Elsner와 Klaus Roehler의 아들이다.

Self-Ask · 논문의 오류 사례

단일 후속 질문

Oskar Roehler의 어머니는 누구인가?

원문의 표현과 잘 맞지 않는 검색

질문: “어머니” / 원문: “두 사람의 아들”

⊗ 해당 사례에서 목표 근거 검색 실패

PIKE-RAG · 논문의 해결 경로

여러 표현의 원자 질의 제안

부모는 누구? · 어머니는 누구? · 가족 정보는?

원자 질문 색인으로 표현 차이 연결

“부모는 누구인가?”에서 원문 청크에 도달

✓ 부모 관계를 담은 근거 확보
어머니: Gisela Elsner

원자 질문을 검색의 중간 연결점으로 사용해, 질문과 문서의 문장 형태 차이를 줄입니다.

출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 17 / PDF 33쪽

한글 재구성 · 원문 번호 유지

그림 17 사례 B · 질문과 원문의 표현 차이

그림 17에서 좌우 경로의 갈림길은 원자 질문의 개수다. 이 방법이 제안하는 원자 질문 중 어머니를 곧바로 묻는 것은 같은 검색 문제를 만나지만, 부모를 묻는 다른 원자 질문은 대상 청크를 성공적으로 끌어온다. 질문 표현을 여럿으로 넓혀 두면 그중 하나가 지식베이스의 서술 구조와 맞물릴 확률이 커진다. 어머니라는 낱말과 아들이라는 낱말 사이의 표현 패턴 차이를, 질문 목록이라는 완충지로 건너는 셈이다. 원자 질문을 검색 단위로 삼는 설계가 여기서 빛을 발한다. 청크를 직접 찾는 대신 청크가 답할 수 있는 질문을 먼저 찾기 때문이다.

사례 C. 원문 문맥을 끝까지 보존

세 번째 사례는 중간 답만 남기는 습관이 만드는 손실이다. Self-Ask는 각 라운드에서 검색한 청크를 중간 답으로 압축한 뒤 청크를 버린다. 그런데 압축 과정에서 주변 맥락이 떨어져 나가면, 그 맥락이 다음 라운드에 필요해질 때 아무것도 남아 있지 않다. 그림 18의 사례에서 Self-Ask는 목표 청크를 검색하는 데는 성공했지만 문맥 정보가 과도하게 많아 Ernie Watts라는 인물을 제대로 가려 내지 못했고, 중간 답이 틀린 경로로 이끌어 최종 결론까지 어긋났다.

사례 C · 원문 문맥을 끝까지 보존

짧은 중간 답만 남기지 않고, 선택한 청크의 풍부한 근거를 유지

원래 질문 · 논문 사례

Ernie Watts는 어느 지역에서 태어났는가?

Self-Ask · 논문의 오류 사례

검색 문서에서 다른 인물을 고름
축구선수 문서가 있었지만 음악가로 해석



중간 답변만 남김
검색한 청크는 다음 문맥에 유지하지 않음



⊗ 잘못된 인물의 출생지로 진행 → 오답

PIKE-RAG · 논문의 해결 경로

간결한 원자 질문 목록에서 선택
관련된 축구선수 문서의 원자 질문을 고름



선택한 원문 청크 전체를 보존
인물 식별과 출생지 근거를 다음 단계에도 유지



✓ 올바른 근거를 재사용
논문에서 보고한 답변: West Berkshire

짧은 중간 답만 유지하지 않고, 선택한 원문 청크를 후속 검색·추론의 문맥에 남깁니다.

출처: Wang 외 · PIKE-RAG · arXiv:2501.11551v4 (2025-03-12) · 원문 그림 18 / PDF 34쪽

한글 재구성 · 원문 번호 유지

그림 18 사례 C · 원문 문맥을 끝까지 보존

그림 18의 오른쪽이 이 방법의 이중 이득을 보여준다. 첫째, 원자 질문 목록을 청크 내용의 요약 후보로 제시하므로 관련 정보를 가려 내는 일이 단순해지고 토큰 사용도 줄어든다. 둘째, 선택된 청크를 중간 답만 남기지 않고 통째로 보존하므로 후속 라운드에 풍부한 문맥이 그대로 쓰인다. 실제로 이 라운드의 원자 질문이 Ernie Watts의 역할을 묻고 있을 때, 그의 출생지 정보는 선택된 청크 안에 이미 들어 있으므로 따로 물어볼 필요가 없다. 청크가 살아 있는 한 그 안의 지식은 다음 라운드에 재사용된다.

세 사례를 관통하는 원칙은 겸손이다. 질문을 이해했다는 확신을 뒤로 미루고, 원문의 표현 방식에 맞춰 질문을 여러 개 준비하고, 한 번 찾은 문맥을 아끼지 않는 태도가 정확도를 만든다. 다음 장에서는 이 원칙을 한국 개발 현장의 언어로 옮겨본다.

DAY

14

한국 IT 현장에서 읽는 PIKE-RAG



이 장부터 관례를 하나 바꾼다. 앞 장들까지는 논문이 말하는 바를 충실히 옮겼다면, 이 장은 필자가 원문의 개념을 한국 개발 현장의 언어로 옮긴 각색이며 원문 저자의 주장이 아니다. 제약 회사 문서를 예로 든 원문의 사례를 사내 위키와 장애 리포트로 바꾸어 읽고, 지식 원자화라는 설계가 우리 조직에서 요구하는 대가까지 함께 따져 본다. 논문의 발상을 현장에 접붙이는 시도로 읽어 주되, 어디까지나 필자의 해석이라는 점을 먼저 밝혀 둔다.

우리 회사 지식베이스도 스캔 문서만 못하다

논문이 지정한 지식 원천의 다양성은 의약품 승인 문서에만 해당하는 이야기가 아니다. 사내 위키에 흩어진 운영 가이드, 규정집, 장애 리포트, API 문서가 우리의 FDA 문서다. 이 문서들의 핵심 정보는 문장으로 쓰여 있지 않은 경우가 많다. 온콜 절차는 표에, 아키텍처는 다이어그램에, 장애 경과는 스크린샷에 붙어 있다. 그런데 벡터 검색 파이프라인은 텍스트 청크만 잘라서 색인하므로, 표의 마지막 행과 다이어그램의 화살표가 검색 대상에서 통째로 빠진다. 장애가 났을 때 온콜 엔지니어가 검색으로 찾지 못하는 문서가 존재하는 이유가 대개 여기에 있다. 청킹 이전에 문서를 해부하는 단계가 없으면, 우리가 가진 지식베이스는 실제로 존재하는 지식베이스보다 작게 동작한다.

질문 네 유형은 백로그 우선순위 도구다

사실 질문, 연결 추론 질문, 예측 질문, 창의 질문으로 나눈 분류를 이 책에서는 논문 이해의 지도로 읽었다. 현장에서는 이 분류를 반대 방향으로 쓸 수 있다. 사내 RAG 백로그의 우선순위를 정하는 도구로 쓰는 것이다. 지원팀에서 하루에 몇백 번씩 던지는 질문 대부분은 규정 조항을 찾아 주는 사실 질문이다. 여기에 정확히 답하는 것만으로도 물음의 절반이 사라진다. 다음은 배포 가이드와 결제 정책을 잇는 연결 추론 질문이다. 이 둘은 이미 쓰인 지식의 범위 안에 있다. 반면 트래픽 추이를 근거로 용량을 예측하는 예측 질문과 아키텍처 개선안을 브레인스토밍하는 창의 질문은 시계열 정리나 도메인 논리 같은 별도 장치를 요구한다. 질문 유형별로 답할 준비가 된 정도가 다르므로, 유형별 물음량을 한 번 세어 보면 어디를 먼저 고쳐야 하는지 백로그가 저절로 정렬된다. 지원 채널의 질문 로그를 유형별로 태깅해 보는 일이 첫 스프린트가 되어야 하는 이유다.

L0에서 L4까지는 로드맵이다

다섯 단계 능력 모델 역시 평가 틀로만 읽을 필요가 없다. L0 지식베이스 구축부터 L1 사실 응답, L2 연결 추론, L3 예측, L4 창의 지원까지를 도입 로드맵으로 환산하면 분기 계획이 된다. 첫 분기는 문서 해부와 정돈이다. 표와 다이어그램을 텍스트로 만들어 회수하고, 출처와 버전을 달아 둔다. 다음 분기는 사실 질문에 답하는 범위를 안정화하는 일이다. 이 단계에서 욕심을 부려 예측이나 창의 기능을 없으면 답이 출처를 이탈하는 사고가 터진다. 논문이 단계별 발전을 강조한 이유가 현장에서는 장애 예방으로 읽힌다. 각 단계의 완성 판정을 내릴 수 있어야 다음 단계로 올라갈 자격이 생긴다.

원자화는 공짜가 아니다

지식 원자화를 비판적으로 읽어야 할 지점도 있다. 청크가 답할 수 있는 원자 질문을 뽑아 색인을 만드는 일은 한 번 끝나는 프로젝트가 아니다. 위키 문서는 매주 고쳐 쓰이고, 규정은 개정되고, API 문서는 버전이 올라간다. 원문서 원자 질문 색인이 문서와 함께 갱신되지 않으면 검색 품질은 소리 없이 썩는다. 살아있는 자산에는 관리 주체가 필요하다는 뜻이다. 문서 작성자가 원자 질문까지 붙이도록 템플릿에 넣는 방법, 자동 생성 후 사람이 검수하는 방법, 분기마다 색인 신선도를 점검하는 방법 중 무엇을 택하든 비용은 어딘가에서 지불된다. 도입 검토서에 검색 정확도의 기대치만 쓰고 이 유지보수 비용을 빠뜨리면, 반년 뒤에 아무도 믿지 않는 챗봇이 된다. 논문이 분해기를 파인튜닝해 비용을 낮추는 길을 제시한 것도 같은 문제의식의 연장으로 읽힌다.

정확히 일치하는 답과 의미가 맞는 답

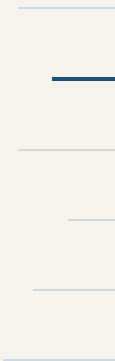
마지막으로 평가 지표에 대한 환기를 하나 남긴다. 법률 벤치마크 장에서 본 것처럼 같은 실험인데 지표에 따라 점수 갭이 몇 배로 벌어질 수 있다. 정확히 일치하는 답만 점수로 주는 EM과 의미가 맞으면 통과시키는 Acc는 서로 다른 것을 낸다. 사내 RAG 평가에서 자동 채점 스크립트 하나로 끝내겠다고 하면, 우리는 보통 EM 쪽에 서게 된다. 그러면 표현이 다를 뿐 맞는 답이 틀린 답으로 잡히고, 그 왜곡된 점수를 근거로 모델을 교체하는 일이 벌어진다. 판정 기준을 정하는 일 자체가 설 계다. 사내 RAG의 성공 지표를 고를 때 무엇을 재고 무엇을 놓치는지 먼저 물어야 하는 까닭이다.

각색은 여기까지다. 다음 장에서는 논문의 결론과 이 책 전체를 한꺼번에 정리하며 맺는다.

DAY

15

맺음말



한 권을 다 읽었으니 논문이 남기는 결론부터 정확히 짚고 넘어가자. 저자들은 산업 현장의 다양한 난제에 맞서려면 검색 증강 생성 시스템의 토대가 전통적 검색 장치를 넘어 특화 지식과 근거를 효과적으로 만들고 활용하는 쪽으로 확장되어야 한다고 말한다. 이 큰 주장은 세 축으로 서 있다.

첫 축은 과제 난이도 기반 분류 패러다임이다. 지식 추출과 이해와 활용의 어려움에 따라 과제를 나누면 시스템 설계와 평가를 위한 새로운 틀이 얻어지고, 단계별로 능력을 탐색하며 알고리즘을 다듬고 응용을 순차적으로 배포할 수 있다. 두 번째 축은 특화 지식과 근거를 중심에 놓는 프레임워크다. 특화 지식을 추출하고 이해하고 정리하며, 원래 질문과 쌓인 지식 위에서 일관된 근거를 세워 정확한 답으로 이끈다. 요구 수준에 맞춰 능력을 조정할 수 있는 것이 이 설계의 이점이다. 세 번째 축은 실증이다. 지식 원자화와 지식 인지형 과제 분해가 멀티홉처럼 복잡한 질문을 다루게 하고, 오픈 도메인과 법률 벤치마크 여러 곳에서 의미 있는 성능 향상을 냈다.

이 세 축은 이 책에서 밟아 온 길과 겹친다. 2장의 질문 유형표를 떠올려 보자. 사실 질문, 연결 추론 질문, 예측 질문, 창의 질문은 답이 지식베이스 안에 드러나 있는가에서 출발해 사실 확인에서 연결 추론으로, 다시 예측과 창안으로 나아가는 사다리였다. 3장의 프레임워크 모듈 지도도 함께 그려 보자. 파일 파싱에서 지식 추출과 저장, 검색과 조직, 지식 중심 추론, 과제 분해와 조율까지 일곱 모듈이 맞물려 돌아가는 그림이었다. 다섯 단계 능력 모델은 이 모듈들을 계단에 얹은 구성표였고, 11장과 12장의 벤치마크 수치는 그 구성이 실제로 작동한다는 영수증이였다. HotpotQA, 2WikiMultiHopQA, MuSiQue의 멀티홉 점수와 LawBench, Open Australian Legal QA의 정확도가 영수증의 두 면이다.

끝으로 다음 독자에게 한마디 하겠다. 이 논문의 진짜 기여는 새 모듈 이름을 몇 개 만든 데 있지 않다. 검색을 잘한다는 질문에서 벗어나, 우리 지식베이스가 무엇을 담고 그 지식이 어떤 질문에 답할 수 있는지를 묻게 만든 데 있다. 문서를 청크로 자르기 전에 그 청크가 답할 수 있는 질문을 먼저 적어 두는 습관, 질문의 의도를 이르게 굳히지 않고 후보를 여럿 두는 습관, 한 번 찾은 문맥을 버리지 않는 습관. 이 세 가지만 들고 가도 검색 증강 시스템을 다루는 눈이 달라질 것이다. 원문은 공개되어 있으니 수치와 도식이 궁금해지면 언제든 펴서 대조하기 바란다.